

Desenvolupament d'un agent de Deep Reinforcement Learning per
a la presa de decisions estratègiques en entorns estocàstics
d'informació imperfecta: El cas de Pokémon VGC

Gerard Pascual Fontanilles

2 de juny de 2026

Índex

1	Introducció	4
1.1	Objectius i Preguntes de Recerca	4
1.2	Contribucions del Treball	5
1.3	Estructura de la Memòria	5
2	Marc Teòric i Contextualització	1
2.1	Contextualització de la IA en Jocs	1
2.1.1	L'Evolució a través de la Informació Perfecta	1
2.1.2	El Salt als Videojocs Moderns	1
2.1.3	Informació Perfecta vs. Informació Imperfecta	2
2.1.4	Processos de Decisió de Markov (MDP i POMDP)	2
2.2	Pokémon: El Joc	4
2.2.1	Orígens i Franquícia	4
2.2.2	Variables de Combat: Estadístiques (Stats)	5
2.2.3	Taula de Tipus, STAB i Categories d'Atacs	6
2.2.4	Velocitat i Ordre d'Execució	8
2.2.5	Condicions d'Estat (Status Conditions)	8
2.2.6	L'Estocasticitat: El Factor d'Incertesa	9
2.3	El Format Competitiu: VGC	9
2.3.1	Orígens i Història	9
2.3.2	En què es diferencia del Pokémon “normal”	9
2.3.3	El Metagame i les Generacions	10
2.4	L'Entorn Virtual: Eines de Simulació	10
2.4.1	Pokémon Showdown	10
2.4.2	Poke-env	11
2.5	Intel·ligència Artificial i Tècniques Computacionals	11
2.5.1	Algorismes de Cerca Clàssics	11
2.5.2	Xarxes Neuronals Artificials	12
2.5.3	Reinforcement Learning (RL)	12
2.5.4	Models de Llenguatge de Gran Escala (LLMs)	14
2.6	Estat de l'Art: Intel·ligència Artificial Aplicada a Pokémon	15
2.6.1	Investigació Inicial: Poke-env i Agents Heurístics	15
2.6.2	Reinforcement Learning i Xarxes Neuronals	15
2.6.3	Models de Llenguatge: Pokechamp i PokeLLMon	16
2.6.4	Connexió amb Altres Dominis d'Informació Imperfecta	16
2.6.5	Buit de la Recerca	16
2.7	Justificació i Posicionament de la Recerca	17
2.7.1	Per què Pokémon és un Cas d'Estudi Ideal	17
2.7.2	Pokémon com a Laboratori d'IA	17
2.8	Síntesi i Pont vers la Metodologia	18

3	Metodologia: Disseny Experimental	19
3.1	Entorn de Simulació	19
3.1.1	Arquitectura Client-Servidor de Pokémon Showdown	19
3.1.2	Poke-env: La Capa d'Abstracció Python	21
3.1.3	Discrepàncies Tècniques: Poke-env vs Pokechamp	22
3.1.4	Variables d'Estat Disponibles	23
3.1.5	Necessitat d'Infraestructura Paral·lela	23
3.1.6	Disseny de la Matriu d'Avaluació (Benchmark Matrix)	24
3.2	Complexitat del Problema	24
3.2.1	Diferències entre Generacions	24
3.2.2	Selecció de Generacions i Verificació de Formats	25
3.3	Espai d'Estats i Espai d'Accions	25
3.3.1	Espai d'Estats (S)	25
3.3.2	Espai d'Accions (A)	26
3.4	Mètriques d'Avaluació	26
3.4.1	Mètriques Primàries	26
3.4.2	Mètriques Secundàries	26
3.5	Disseny dels Experiments	27
3.5.1	Tipus de Matchups	27
3.5.2	Formats i Condicions	27
3.5.3	Agregació de Resultats	27
3.5.4	Propera Fase: Anàlisi de la Distribució de Pokémon	28
3.6	Fases de Desenvolupament i Models de Referència	28
3.6.1	Fase 1: Disseny de la Lògica de Decisió Heurística	28
3.6.2	Fase 2: Models d'Aprenentatge Profund i Raonament per LLM	29
3.6.3	Línies de Control (Baselines)	29
4	Entorn de Desenvolupament i Infraestructura de Computació	31
4.1	Evolució Cronològica de l'Arquitectura	31
4.1.1	Fase 1: Desenvolupament Local i Prototipatge (Node Client)	31
4.1.2	Fase 2: Consolidació en Servidor de Torre (Node Remot)	31
4.2	Gestió de Dependències i Entorn Python: uv	32
4.3	Seguretat i Connectivitat Remota	32
4.4	Metodologia i Control de Versions	33
4.5	Qualitat de Codi i Eines de Suport	33
5	Implementació d'Agents: De les Fases Heurístiques a l'Aprenentatge Profund	34
5.1	Contribucions de Software del Projecte	34
5.2	Arquitectura Master-Worker per a Benchmarking Massiu	35
5.2.1	Visió General del Patró	35
5.2.2	Paral·lisme i Thread-Safety	35
5.2.3	Gestió de Memòria: RAM vs Disc	36
5.2.4	Tolerància a Fallades (Crash-Safe State)	37
5.2.5	Reinici Periòdic de Servidors	37
5.2.6	Paràmetres de Configuració	38
5.2.7	Dades Recollides per Partida	38
5.2.8	Instrumentació: StatsBattle	39
5.2.9	Workflow Complet d'una Execució	39
5.3	Fase 1.1: Desenvolupament d'Heurístiques per Singles (1vs1)	40
5.4	Fase 1.2: Integració d'Agents Externs (Pokechamp)	41
5.4.1	Agents Heurístics de Pokechamp (1vs1)	41
5.4.2	Agents LLM: Pokechamp i PokeLLMon	41

5.5	Fase 1.2b: L'Ascens al Competitiu Real (Dobles VGC)	42
5.5.1	Evolució i Refinament de les Heurístiques de Dobles	42
5.6	Pipeline de Processament de Dades i Analítica	43
5.7	Fase 2: Implementació del Pipeline de Reinforcement Learning	43
5.7.1	Vectorització de l'Estat (StateDUMMY_UNDERSCOREVectorizer)	44
5.7.2	Entorn Gymnasium amb Action Masking	44
5.7.3	Reward Shaping (Disseny de la Recompensa)	44
5.7.4	Curriculum d'Entrenament en 4 Fases	45
5.7.5	Arquitectura del Model i Infraestructura GPU	45
5.8	Treball Futur: Aprenentatge Basat en Datasets de Partides	46
5.8.1	Full de Ruta Immediat: Behavioral Cloning	46
6	Resultats i Avaluació de les Heurístiques	47
6.1	Infraestructura d'Execució: Gestió de Recursos	47
6.2	Avaluació Exhaustiva al Format 1 vs 1 (Singles)	48
6.2.1	Anàlisi de Metagames i Dinàmiques de Joc	51
6.2.2	Freqüència de Moviments i Popularitat de Pokémon	51
6.3	Avaluació al Format 2 vs 2 (Doubles)	52
7	Conclusions	56
7.1	Resum de Contribucions	56

Capítol 1

Introducció

La recerca en Intel·ligència Artificial aplicada a jocs ha evolucionat des de clàssics d'informació perfecta, com els escacs o el Go, fins a videojocs moderns d'alta complexitat i informació parcial. En aquest context, Pokémon —i en particular el seu format competitiu VGC— esdevé un banc de proves privilegiat per estudiar agents capaços de prendre decisions sota incertesa, amb dinàmiques estocàstiques i espais d'estat i d'accions combinatoris.

Aquest Treball de Final de Màster se centra en el desenvolupament i avaluació d'agents racionals capaços de competir en batalles de Pokémon utilitzant el simulador de codi obert *Pokémon Showdown* [13] i la llibreria *poke-env* [17]. El problema es modelitza com un Procés de Decisió de Markov Parcialment Observable (POMDP) [21], on l'agent només disposa d'una observació parcial de l'estat real i ha de gestionar múltiples fonts d'aleatorietat (dany variable, precisió dels moviments, cops crítics, condicions d'estat).

1.1 Objectius i Preguntes de Recerca

L'objectiu general del TFM és avaluar fins a quin punt és possible construir agents d'IA competitius per a Pokémon, combinant heurístiques de tipus expert amb algorismes de Deep Reinforcement Learning (DRL). A partir d'aquest objectiu general, es plantegen els objectius específics següents:

- Dissenyar i implementar una infraestructura escalable de simulació basada en *Pokémon Showdown* i *poke-env*, capaç d'executar centenars de batalles en paral·lel de manera robusta.
- Desenvolupar una família d'agents heurístics (basats en regles) per al format *Singles* (1 vs 1) que capturin coneixement expert sobre Taula de Tipus, STAB, velocitat, prioritat i gestió de riscos.
- Construir un pipeline complet de Reinforcement Learning per entrenar agents neuronals sobre l'entorn de *poke-env*, incloent la vectorització de l'estat, el disseny de la recompensa i l'*action masking*.
- Integrar i orquestrar baselines externs (agents nadius de *poke-env* i agents heurístics/LLM de Pokechamp [12]) per disposar d'un marc comparatiu sòlid.
- Avaluar quantitativament el rendiment dels agents propis respecte aquestes línies base, analitzant tant la taxa de victòries (Win-Rate) com mètriques secundàries de qualitat de joc.

De forma operacional, això es pot resumir en la pregunta central:

Pot un agent de Reinforcement Learning, entrenat sobre un estat vectoritzat de Pokémon i amb accions discretitzades, superar de manera consistent heurístiques expertes dissenyades a mà i baselines estàndard de la literatura?

1.2 Contribucions del Treball

Les contribucions principals d'aquest TFM es poden agrupar en tres eixos complementaris:

1. **Infraestructura de simulació i orquestració paral·lela:** S'ha implementat un orquestrador propi capaç de llançar múltiples instàncies locals de *Pokémon Showdown* en ports consecutius, gestionar-les de forma asíncrona amb Python (**asyncio**) i mantenir checkpoints de progrés. Aquesta infraestructura permet executar centenars de batalles simultànies, controlant fuites de memòria i errors de connexió.
2. **Agents heurístics experts per a Singles:** S'ha desenvolupat una família d'heurístiques internes (v1-v6) que integren càlcul de dany amb Taula de Tipus, STAB, categoria física/especial, condicions de camp, boosts d'estadístiques, prioritat i regles de substitució defensiva. Aquest conjunt d'agents defineix un "límit de control expert" contra el qual mesurar futurs models neuronals.
3. **Pipeline de Reinforcement Learning amb action masking:** S'ha implementat un entorn compatible amb Gymnasium que vectoritza l'estat de la batalla en un vector numèric de 238 dimensions i defineix un espai d'accions discret amb màscara d'accions il·legals. Sobre aquest entorn s'ha entrenat un agent *MaskablePPO* [16], amb reward shaping específic per al domini Pokémon i un currículum progressiu d'oponents.

A més, el treball integra de forma sistemàtica baselines nadius de *poke-env* (RandomPlayer, MaxBasePowerPlayer, SimpleHeuristicPlayer) i agents del repositori Pokechamp (Abyssal, SafeOneStep, Pokechamp i PokeLLMon) com a rivals controlats. Aquests agents externs no constitueixen una contribució pròpia, però sí un element central del marc experimental.

1.3 Estructura de la Memòria

La resta del document s'organitza de la manera següent:

- El **Capítol 2** presenta el marc teòric i de context: evolució de la IA en jocs, mecàniques de Pokémon i VGC, formalització en termes de MDP/POMDP, tècniques de Reinforcement Learning i agents existents com Pokechamp.
- El **Capítol 3** descriu la metodologia experimental: l'entorn de simulació, la definició formal dels espais d'estats i accions, les mètriques d'avaluació (amb especial èmfasi en el Win-Rate) i el disseny del conjunt d'experiments.
- El **Capítol 4** detalla l'entorn de computació i la infraestructura de desenvolupament remot, incloent la seguretat amb Tailscale i el flux de treball basat en Remote-SSH.
- El **Capítol 5** detalla la implementació pràctica dels agents: arquitectura de software, evolució de les heurístiques, integració d'agents externs i construcció del pipeline de Reinforcement Learning.
- El **Capítol 6** (Resultats) presenta els experiments realitzats i discuteix quantitativament el rendiment dels diferents agents.
- Finalment, el treball es tanca amb un capítol de conclusions i línies de treball futur, on es resumeixen els principals resultats i es proposen extensions com l'aprenentatge basat en datasets de partides competitives.

Resum

Els videojocs d'estratègia complexos constitueixen un banc de proves valuós per a la recerca en Intel·ligència Artificial. En contrast amb jocs d'informació perfecta (com els escacs o el Go), els entorns amb informació parcial i dinàmiques estocàstiques presenten reptes addicionals significatius en predicció i planificació. Aquest Treball de Final de Màster proposa el disseny, implementació i avaluació d'un agent basat en Deep Reinforcement Learning (DRL) per al format competitiu VGC de Pokémon, utilitzant el simulador de codi obert Pokémon Showdown [13] i la llibreria poke-env [17] com a entorn d'entrenament.

El problema s'aborda com un Procés de Decisió de Markov Parcialment Observable (POMDP) [21], caracteritzat per un espai d'estats i d'accions d'alta dimensionalitat i múltiples fonts d'incertesa (informació oculta sobre l'equip rival i resultats estocàstics dels moviments). L'objectiu no és la resolució perfecta del joc, sinó l'aproximació d'una política d'actuació que maximitzi l'esperança de victòria sota incertesa.

La proposta integra: (1) una representació vectorial de l'estat que inclou estadístiques, tipus, condicions de camp i historial d'accions; (2) una arquitectura neuronal (p. ex., xarxa recurrent o encoder + policy network) per captar dependències temporals i la parcialitat de la informació; i (3) estratègies d'entrenament basades en self-play i algorismes actor-crític (com PPO [18]), complementats potencialment amb components d'aprenentatge supervisat per a la predicció de moviments rivals.

L'avaluació es durà a terme en un entorn de simulació local controlat. S'analitzarà el rendiment quantitatiu de l'agent (taxa de victòries i recompensa mitjana acumulada) enfront d'una bateria d'agents basats en regles (scripted baselines) que implementen heurístiques competitives estàndard. Addicionalment, es realitzaran estudis d'ablació (ablation studies) per determinar la contribució de cada mòdul de l'arquitectura. Qualitativament, s'examinarà l'emergència de comportaments avançats com la predicció de canvis de Pokémon (switch prediction) i la gestió del risc, discutint finalment la viabilitat computacional i les limitacions del model proposat.

Paraules clau: Deep Reinforcement Learning (DRL), POMDP, Teoria de Jocs, Sistemes Estocàstics, Pokémon VGC, Xarxes Neuronals.

Capítol 2

Marc Teòric i Contextualització

Aquest capítol estableix els fonaments teòrics necessaris per comprendre el desenvolupament d'un agent d'intel·ligència artificial aplicat a entorns complexos. S'introdueixen els conceptes clau sobre teoria de jocs i IA, es presenta Pokémon i el seu format competitiu VGC com a cas d'estudi, es descriuen les eines de simulació disponibles, es defineixen les tècniques modernes d'aprenentatge automàtic que sustenten el projecte, es revisa l'estat de l'art en IA aplicada a Pokémon, i es justifica el posicionament de la recerca dins la literatura existent.

2.1 Contextualització de la IA en Jocs

2.1.1 L'Evolució a través de la Informació Perfecta

Històricament, els jocs han servit com a banc de proves privilegiat per avaluar les capacitats de la Intel·ligència Artificial (IA). Els denominats jocs d'**informació perfecta** — on tots els jugadors coneixen completament l'estat del joc en tot moment — han protagonitzat les fites més emblemàtiques.

Deep Blue (IBM, 1997) va derrotar el campió mundial d'escacs Garry Kasparov emprant l'algorisme *Minimax* amb esporgada alfa-beta [8, 10]. L'estratègia es basava en explorar milions de posicions possibles per torn dins un arbre de cerca, podant les branques que no podien millorar el resultat conegut. En els escacs, aquesta exploració és viable perquè tot l'estat del joc és visible sobre el tauler.

Dues dècades més tard, **AlphaGo** (Google DeepMind, 2016) va aconseguir derrotar Lee Sedol, un dels millors jugadors de Go del món [20]. El Go presenta un factor de ramificació incomparablement superior als escacs (~ 250 moviments legals per torn davant els ~ 35 dels escacs), la qual cosa fa inviable una exploració exhaustiva. Per superar-ho, AlphaGo va combinar la *Cerca d'Arbre de Monte Carlo* (MCTS) [5] amb xarxes neuronals profundes entrenades per *self-play*. La seva evolució, **AlphaZero** [19], va generalitzar l'aproximació per dominar escacs, shogi i Go simultàniament sense coneixement humà previ, aprenent únicament de jugar contra si mateixa.

2.1.2 El Salt als Videojocs Moderns

Les fites anteriors es circumscriuen a jocs de tauler d'informació perfecta i amb regles estàtiques. El gran salt posterior de la IA ha estat conquistar **videojocs moderns**, entorns que es caracteritzen per:

- **Espais d'acció enormes i continus:** centenars d'accions possibles per segon.
- **Informació parcial:** parts del mapa o les accions rivals queden ocultes.
- **Múltiples agents simultanis:** cooperació i competició entre 2–10 jugadors.

- **Horitzons temporals llargs:** partides de 20–60 minuts amb milers de decisions seqüencials.

AlphaStar (DeepMind, 2019) va assolir el nivell Gran Mestre a *StarCraft II*, un joc d'estratègia en temps real d'informació imperfecta. El sistema combinava aprenentatge per imitació sobre partides humanes amb *multi-agent reinforcement learning*, mantenint una lliga d'agents interns que evolucionaven competint entre ells [28].

OpenAI Five (2019) va derrotar el campió mundial del videojoc *Dota 2* en format 5 contra 5. Cada agent controlava un heroi amb més de 170.000 observacions possibles i desenes d'accions per segon, cooperant en equip durant partides de 45 minuts [1].

Fins i tot en jocs d'atzar com el *Pòquer*, sistemes com **Libratus** [3] i **Pluribus** [4] han superat els millors jugadors professionals en la variant *No-Limit Texas Hold'em*, on la informació oculta (les cartes dels rivals) és l'element central de l'estratègia.

2.1.3 Informació Perfecta vs. Informació Imperfecta

Les fites anteriors es poden classificar segons el grau d'observabilitat de l'entorn:

- En un joc d'**informació perfecta** (escacs, Go), l'estat complet del sistema és visible per tots els jugadors en tot moment. Les decisions es basen exclusivament en l'anàlisi de l'estat actual i la predicció de les accions futures del rival.
- En un joc d'**informació imperfecta** (pòquer, StarCraft, Pokémon), part de l'estat roman oculta. Els jugadors han de *deduir* informació a partir d'observacions parcials i gestionar la **incertesa** inherent.

Quan, a més, el resultat de les accions no és determinista sinó que depèn de variables aleatòries (probabilitat de colpejar, dany variable, etc.), es parla d'un entorn **estocàstic**. La combinació d'informació imperfecta i estocasticitat crea entorns extremament desafiants per a la IA.

2.1.4 Processos de Decisió de Markov (MDP i POMDP)

Formalment, la presa de decisions seqüencials en entorns estocàstics es modelitza amb un **Procés de Decisió de Markov (MDP)** [24]. Un MDP estableix que les conseqüències d'una acció depenen únicament de l'estat *actual* del sistema, no de tota la seva història (propietat de Markov). Es defineix com una tupla (S, A, T, R, γ) :

- S : Conjunt d'estats possibles.
- A : Conjunt d'accions disponibles.
- T : Funció de transició, $P(s' | s, a)$, que indica la probabilitat d'arribar a l'estat s' des de s fent l'acció a .
- R : Funció de recompensa, $R(s, a)$, que quantifica el benefici immediat d'una decisió.
- γ : Factor de descompte ($\gamma \in [0, 1)$), que pondera la importància de les recompenses futures.

Un exemple clàssic d'MDP seria un robot navegant per una quadrícula: coneix la seva posició exacta, les seves accions (moure's en 4 direccions) i les recompenses (arribar a l'objectiu). L'objectiu de l'agent és trobar la **política** $\pi(a | s)$ que maximitza la recompensa acumulada esperada.

Tanmateix, si l'agent **no pot observar** l'estat complet del sistema, el model s'estén a un **Procés de Decisió de Markov Parcialment Observable (POMDP)** [21]. Un POMDP afegeix dos elements:

- Ω : Conjunt d'observacions que rep l'agent (una versió parcial o sorollosa de l'estat real).
- O : Funció d'observació, $P(o | s', a)$, que condiciona quines observacions arriben a l'agent.

En un POMDP, l'agent construeix i manté un *belief state*: una distribució de probabilitat sobre tots els estats possibles, actualitzada a cada observació rebuda. Per exemple, un jugador de pòquer no veu les cartes dels rivals, però dedueix probabilitats sobre les seves mans a partir de les apostes observades.

La Figura 2.1 il·lustra la diferència fonamental entre ambdós models: mentre que en un MDP l'agent rep l'estat complet del sistema, en un POMDP només accedeix a observacions parcials i ha d'inferir l'estat ocult.

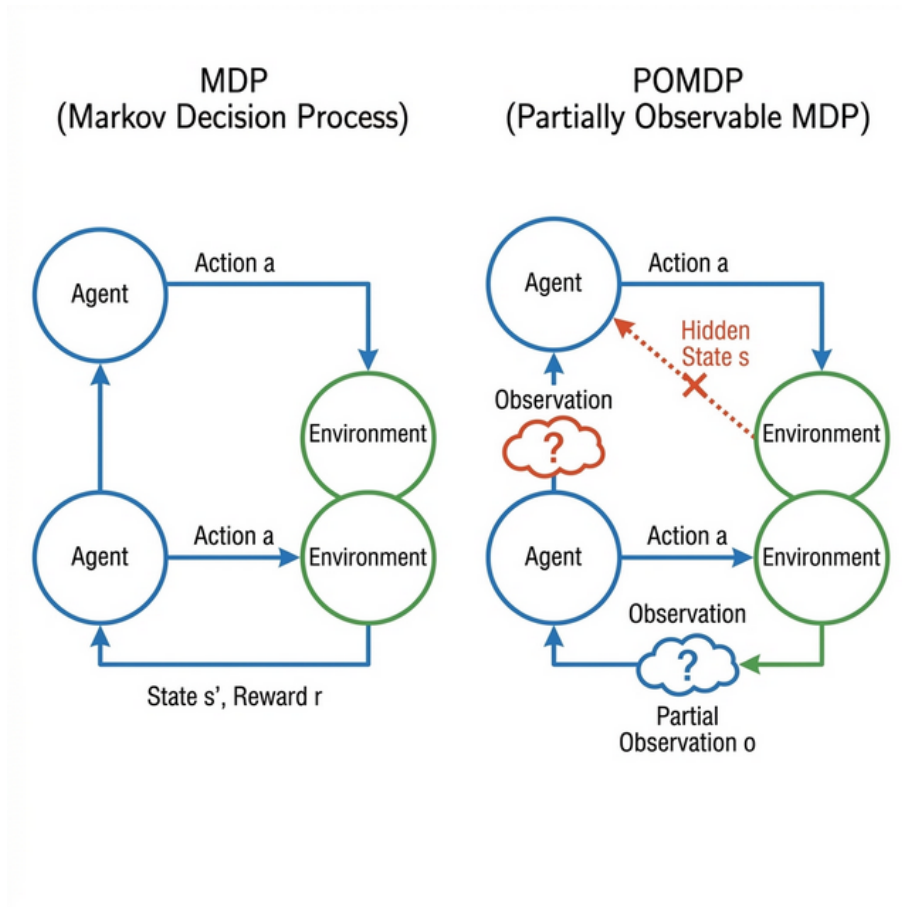


Figura 2.1: Comparació entre un MDP (esquerra) i un POMDP (dreta). En el MDP, l'agent observa l'estat complet s . En el POMDP, l'estat real queda ocult i l'agent només rep observacions parcials o , forçant la construcció d'un *belief state*.

Mapatge Formal: POMDP Aplicat a Pokémon

Per concretitzar el marc abstracte dels POMDPs al nostre domini d'estudi, la Taula 2.1 estableix la correspondència entre cada component formal i la seva instanciació en una batalla Pokémon VGC [kaelbling'pomdp].

Component	Instanciació en Pokémon VGC
S (Estats)	Configuració completa: 6 Pokémon per bàndol (espècie, HP, EVs, moviments, objecte, estat de salut, boosts), efectes de camp actius, torn actual. Espai extremament gran: $ S \gg 10^{100}$.
A (Accions)	Accions legals cada torn: escollir un de ≤ 4 moviments o canviar a un de ≤ 5 Pokémon de reserva. En Dobles, el producte cartesià de les accions de les dues unitats actives. $ A \approx 9$ en Singles, ≤ 81 en Dobles.
T (Transició)	Mecàniques del joc: càlcul de dany (amb factor aleatori $\in [0.85, 1.0]$), aplicació de STAB, Taula de Tipus, prioritat, resolució de velocitat, activació de condicions d'estat. El motor de Pokémon Showdown implementa T amb precisió mil·limètrica.
R (Recompensa)	Senyal inherentment escàs: +1 en victòria, -1 en derrota, 0 durant la batalla. En entrenament amb <i>reward shaping</i> : recompenses intermèdies addicionals.
Ω (Observacions)	Informació parcial rebuda cada torn: Pokémon propis (complet), Pokémon rivals actius (només espècie, HP%, moviments ja observats), efectes de camp. Cruccialment, IVs, EVs, Nature, objecte i moviments no revelats del rival romanen ocults.
O (Observació)	La funció $P(o s', a)$ és majoritàriament determinista (l'agent sempre observa el resultat del moviment), però parcial (no veu les variables internes de l'oponent).

Taula 2.1: Mapatge formal dels components POMDP abstractes a la seva instanciació concreta en Pokémon VGC. S'evidencia que Pokémon constitueix un POMDP genuí amb estat enorme i parcialment observable.

Aquesta formalització il·lustra per què Pokémon és un domini particularment ric per estudiar la presa de decisions sota incertesa: l'estat és vast i parcialment observable, les transicions incorporen múltiples fonts d'estocasticitat, i l'agent ha d'actuar amb informació incompleta. A les seccions següents es detallen les mecàniques específiques del joc que defineixen concretament els espais S , A i Ω , el format competitiu VGC, i les eines de simulació que permeten aquest estudi.

2.2 Pokémon: El Joc

Abans de presentar les tècniques d'Intel·ligència Artificial que sustenten el projecte, és essencial comprendre profundament el domini sobre el qual operen els agents. Les regles formals del combat Pokémon — estadístiques, tipus, velocitat, condicions d'estat i estocasticitat — defineixen concretament els espais S , A i Ω del POMDP formalitzat a la secció anterior. Comprendre aquestes mecàniques és fonamental per apreciar per què certs algorismes són més adequats que altres i quines decisions de disseny s'adopten als capítols posteriors.

2.2.1 Orígens i Franquícia

Pokémon, originat com a contracció del terme japonès *Poketto Monsutā* (*Pocket Monsters*), és la franquícia d'entreteniment més rendible de la història, celebrant el seu 30è aniversari el 2026 (Figura 2.2) [6]. Creat per Satoshi Tajiri i publicat per Nintendo el 1996, el joc original consistia en explorar una regió fictícia amb l'objectiu de capturar criatures, entrenar-les i aconseguir les vuit medalles dels líders de gimnàs per convertir-se en Campió de la Lliga.



Figura 2.2: Logotip commemoratiu del 30è aniversari de la franquícia Pokémon.

El nucli del joc rau en les **batalles per torns** entre Pokémon. Existeixen dos formats principals: **Singles** (1 vs 1, un Pokémon al camp per bàndol) i **Doubles** (2 vs 2, dos Pokémon al camp per bàndol). Sota l'aparença accessible del joc, les batalles amaguen un motor de complexitat matemàtica notable que es detalla a continuació.

2.2.2 Variables de Combat: Estadístiques (Stats)

Cada Pokémon es defineix per sis estadístiques fonamentals que determinen el seu rendiment en combat: Punts de Salut (HP), Atac (físic), Defensa (física), Atac Especial, Defensa Especial i Velocitat. Cada espècie té una distribució base d'stats diferent, especialitzant-la en un rol concret. La Figura 2.3 mostra l'exemple de Pikachu, un Pokémon veloç i ofensiu especialment.

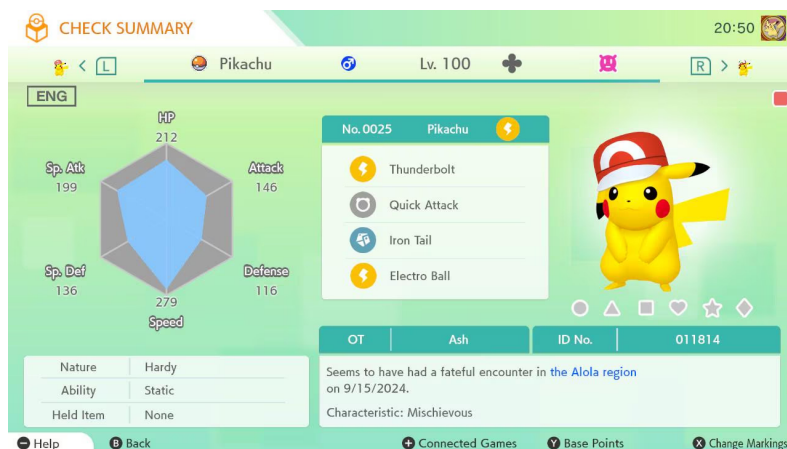


Figura 2.3: Diagrama de les estadístiques base de Pikachu. Es destaquen Velocitat i Atac Especial com a punts forts, amb defenses baixes.

Les estadístiques base es modifiquen per tres factors configurables que constitueixen **informació oculta** durant la batalla:

1. **Individual Values (IVs)**: Valors genètics intrínsecs entre 0 i 31, inherents a cada exemplar. En l'àmbit competitiu, generalment es fixen a 31 per maximitzar el rendiment.
2. **Effort Values (EVs)**: Punts d'entrenament distribuïbles lliurement, amb un límit global de 510 i un màxim de 252 per estadística. Aquesta distribució defineix l'**arquitectura estratègica** del Pokémon i constitueix l'àmbit principal d'engany competitiu, ja que l'oponent desconeix si un Pokémon ha estat construït per atacar, per resistir o per equilibrar-se.

3. **Naturaleses (Natures)**: Modificadors multiplicatius que atorguen un +10% a una estadística i un -10% a una altra, permetent accentuar el perfil desitjat.

La Figura 2.4 il·lustra la relació entre aquests tres components.

	Base	IVs	EVs	
HP	35	1	83	201
Attack	55	1	122	146 --
Defense	40	1	59	100 --
Sp. Atk	50	1	171	148 --
Sp. Def	50	1	67	122 --
Speed	90	1	142	221 --

Current HP: 201 / 201 (100%) Dynamax

Figura 2.4: Exemple de com IVs, EVs i Nature modifiquen les estadístiques base d'un Pokémon.

El valor final de cada estadística (excepte HP) es calcula amb la fórmula oficial [7]:

$$Stat = \left\lfloor \left(\frac{(2 \times Base + IV + \lfloor \frac{EV}{4} \rfloor) \times Level}{100} + 5 \right) \times Nature \right\rfloor \quad (2.1)$$

Per als Punts de Salut, la fórmula varia lleugerament en afegir una constant extra i no aplicar la Nature:

$$HP = \left\lfloor \frac{(2 \times Base + IV + \lfloor \frac{EV}{4} \rfloor) \times Level}{100} \right\rfloor + Level + 10 \quad (2.2)$$

En el format competitiu, tots els Pokémon es normalitzen a *Level* 50, garantint un camp igualitari.

2.2.3 Taula de Tipus, STAB i Categories d'Atacs

L'eix central de la presa de decisions en combat és la **Taula de Tipus** (Taula 2.2). Existeixen 18 tipus elementals (Aigua, Foc, Planta, Elèctric, etc.) que estableixen una matriu de relacions de dany:

- **Super Effective** ($\times 2$): el tipus de l'atac és fort contra el tipus del defensor.
- **Not Very Effective** ($\times 0.5$): el tipus de l'atac és dèbil contra el defensor.

- **Immunitat** ($\times 0$): el defensor és completament immune.

Com que molts Pokémon posseeixen **dobles tipus**, els multiplicadors es combinen, podent generar febleses extremes ($\times 4$) o dobles resistències ($\times 0.25$).

La Taula 2.2 presenta la matriu completa de relacions entre els 18 tipus.

Taula 2.2: Matriu de la Taula de Tipus (Gen. VI–IX). Files = tipus de l'atac, columnes = tipus del defensor. **2** = Super Effective ($\times 2$), $\frac{1}{2}$ = Not Very Effective ($\times 0.5$), **0** = Immunitat ($\times 0$), buit = Neutral ($\times 1$).

	NOR	FOC	AIG	PLA	ELE	GEL	LLU	VER	TER	VOL	PSI	INS	ROC	FAN	DRA	FOS	ACE	FAD
NOR													$\frac{1}{2}$	0			$\frac{1}{2}$	
FOC			$\frac{1}{2}$	$\frac{1}{2}$	2		2					2	$\frac{1}{2}$		$\frac{1}{2}$		2	
AIG		2		$\frac{1}{2}$	$\frac{1}{2}$				2				2		$\frac{1}{2}$			
PLA		$\frac{1}{2}$	2		$\frac{1}{2}$			$\frac{1}{2}$	2	$\frac{1}{2}$		$\frac{1}{2}$	2		$\frac{1}{2}$		$\frac{1}{2}$	
ELE				2		$\frac{1}{2}$			0	2					$\frac{1}{2}$			
GEL		$\frac{1}{2}$	$\frac{1}{2}$	2			$\frac{1}{2}$		2	2					2		$\frac{1}{2}$	
LLU	2					2		$\frac{1}{2}$		$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	2	0		2	2	$\frac{1}{2}$
VER				2				$\frac{1}{2}$	$\frac{1}{2}$				$\frac{1}{2}$	$\frac{1}{2}$			0	2
TER		2		$\frac{1}{2}$	2			2		0		$\frac{1}{2}$	2				2	
VOL				2	$\frac{1}{2}$		2	2				2	$\frac{1}{2}$				$\frac{1}{2}$	
PSI							2	2			$\frac{1}{2}$					0	$\frac{1}{2}$	
INS		$\frac{1}{2}$		2			$\frac{1}{2}$	$\frac{1}{2}$		$\frac{1}{2}$	2			$\frac{1}{2}$		2	$\frac{1}{2}$	$\frac{1}{2}$
ROC		2				2	$\frac{1}{2}$		$\frac{1}{2}$	2		2					$\frac{1}{2}$	
FAN	0										2			2		$\frac{1}{2}$		
DRA															2		$\frac{1}{2}$	0
FOS							$\frac{1}{2}$				2			2		$\frac{1}{2}$		$\frac{1}{2}$
ACE		$\frac{1}{2}$	$\frac{1}{2}$		$\frac{1}{2}$	2							2				$\frac{1}{2}$	2
FAD		$\frac{1}{2}$					2	$\frac{1}{2}$							2	2	$\frac{1}{2}$	

Per il·lustrar-ho amb exemples concrets:

- Un atac *Thunderbolt* (Elèctric) contra un *Squirtle* (Aigua): Elèctric és *Super Effective* contra Aigua $\rightarrow \times 2$ de dany.
- Un atac *Flamethrower* (Foc) contra un *Bulbasaur* (Planta/Verí): Foc és *Super Effective* contra Planta ($\times 2$), però neutre contra Verí ($\times 1$), resultant en $\times 2$ total.
- Un atac *Rock Slide* (Roca) contra un *Charizard* (Foc/Volador): Roca és *Super Effective* contra Foc ($\times 2$) i contra Volador ($\times 2$), resultant en una feblesa extrema de $\times 4$ de dany.
- Un atac *Earthquake* (Terra) contra un *Pidgeot* (Normal/Volador): Terra té **immunitat** contra Volador $\rightarrow \times 0$ de dany, tot i ser normal contra Normal.

Quan un Pokémon utilitza un atac del seu propi tipus, rep el bonus **STAB** (*Same Type Attack Bonus*), un multiplicador addicional de $\times 1.5$ sobre el dany base. Per exemple, quan un *Squirtle* (tipus Aigua) utilitza *Surf* (atac Aigua), el dany es multiplica per $\times 1.5$ automàticament. Combinat amb la Taula de Tipus, un *Surf* amb STAB contra un Pokémon de Foc causaria $1.5 \times 2.0 = \times 3.0$ de dany efectiu.

Els moviments es classifiquen en tres categories:

- **Físics**: utilitzen l'estadística d'Atac de l'atacant contra la Defensa del defensor.
- **Especials**: utilitzen l'Atac Especial contra la Defensa Especial.
- **D'Estat** (*Status*): no causen dany directe. Permeten estratègies com protegir-se (*Protect*), augmentar estadístiques pròpies (*Bufs*), debilitar l'enemic (*Debuffs*), o infligir condicions d'estat.

En el format de Dobles, existeixen a més atacs **d'àrea** (*Spread*) que colpegen tots els rivals al camp simultàniament, però amb una penalització de dany a $\times 0.75$ [22].

2.2.4 Velocitat i Ordre d'Execució

L'ordre en què els Pokémon actuen cada torn és determinat per l'estadística de **Velocitat**: el Pokémon amb un valor més alt ataca primer. Per exemple, si un *Jolteon* (Velocitat base: 130) s'enfronta a un *Snorlax* (Velocitat base: 30), Jolteon actuarà sempre primer en condicions normals. Controlar la velocitat relativa és fonamental: atacar primer pot significar la diferència entre derrotar l'oponent o ser derrotat abans de poder actuar.

Tanmateix, existeixen rangs de **Prioritat** (de -7 a $+5$) que anul·len completament el càlcul de Velocitat. Un moviment de prioritat $+1$ s'executa sempre abans que qualsevol moviment de prioritat 0 , independentment de la Velocitat de qui l'usi. Per exemple:

- *Quick Attack* (prioritat $+1$): Permet a un Pokémon lent com *Snorlax* atacar primer davant d'un *Jolteon* més ràpid, ideal per assegurar un KO contra un oponent amb poca vida.
- *Protect* (prioritat $+4$): S'executa gairebé sempre primer, protegint el Pokémon de tot dany durant un torn. Essencial en el format de Dobles.
- *Trick Room* (prioritat -7): S'executa últim, però inverteix l'ordre de Velocitat durant 5 torns: els Pokémon lents actuen primer. Aquesta mecànica transforma completament les dinàmiques de la partida.

2.2.5 Condicions d'Estat (Status Conditions)

Les condicions d'estat introdueixen disrupcions estocàstiques addicionals al combat, forçant l'agent a adaptar la seva estratègia davant imprevistos. La Taula 2.3 resumeix les principals:

Estat	Efecte en Stats	Mecànica i Exemple
Cremada (Burn)	↓ 50% Atac Físic	Perd 1/16 de la vida per torn. Un <i>Machop</i> cremat perd la meitat de la seva força ofensiva, invalidant el seu rol d'atacant físic.
Paràlisi (Paralysis)	↓ 50% Velocitat	25% de probabilitat de no poder actuar cada torn. Un <i>Jolteon</i> paralytitzat perd el seu avantatge de velocitat i pot quedar incapacitat en moments crítics.
Verí (Poison)	Cap directe	Perd 1/8 de vida per torn; si és verí tòxic, el dany creix exponencialment (1/16, 2/16, 3/16...), penalitzant Pokémon que vulguin mantenir-se al camp molt temps.
Son (Sleep)	Cap directe	El Pokémon no pot actuar durant 1–3 torns aleatoris. Un Pokémon adormit és completament indefens.
Congelació (Freeze)	Cap directe	No pot actuar; 20% de probabilitat de descongelar-se per torn. Potencialment devastador per la seva durada indefinida.

Taula 2.3: Condicions d'estat principals i el seu impacte. L'agent ha de considerar tant la probabilitat d'infligir-les com el seu efecte sobre la presa de decisions.

L'impacte estratègic de les condicions d'estat és significatiu: un agent intel·ligent pot optar per infligir una Cremada a l'atacant físic rival en lloc d'atacar directament, reduint la seva amenaça a llarg termini. Igualment, ha de considerar si val la pena substituir un Pokémon enverinat o mantenir-lo al camp per aprofitar un avantatge de tipus.

2.2.6 L'Estocasticitat: El Factor d'Incertesa

Més enllà de les condicions d'estat, el combat Pokémon incorpora múltiples fonts d'**aleatorietat inherent** que impedeixen tota predicció determinista:

1. **Dany Aleatori (*Damage Roll*)**: Cada atac rep un multiplicador aleatori uniforme dins el rang $[0.85, 1.00]$. Això significa que un mateix atac pot causar fins a un 15% menys de dany del previst, convertint KOs aparentment segurs en fallides.
2. **Precisió dels Moviments (*Accuracy*)**: Alguns atacs potents no tenen un 100% de probabilitat d'encertar (p. ex., *Hydro Pump* té un 80% de precisió). L'agent ha de ponderar l'esperança de dany contra el risc de fallar.
3. **Cops Crítics (*Critical Hits*)**: Amb una probabilitat base de $\sim 4.16\%$ ($1/24$), qualsevol atac pot causar un 50% més de dany, ignorant les modificacions defensives de l'oponent [22].

Aquesta triple font d'aleatorietat fa que **cap estratègia garanteixi un resultat al 100%**: la millor heurística del món pot perdre contra un rival inferior si l'atzar li és desfavorable. Aquesta característica defineix el marge residual irreductible de victòria del qual cap IA podrà escapar.

2.3 El Format Competitiu: VGC

2.3.1 Orígens i Història

El circuit **VGC** (*Video Game Championships*) va ser creat per *The Pokémon Company* l'any 2009 com a format oficial de competició a nivell mundial [26]. Des d'aleshores, centenars de milers de jugadors hi participen anualment en tornejos regionals, nacionals i el *Pokémon World Championships*, amb premis de fins a 30.000\$ per al campió [27].

Figures com **Wolfe Glick** (WolfeyVGC), doble campió del món (2016 i 2019), han contribuït a divulgar l'escena competitiva a través de contingut educatiu accessible [9]. La comunitat d'estratègia competitiva, amb plataformes com *Smogon University* [23], també ha desenvolupat durant dècades un cos ingent d'anàlisi teòrica sobre el metagame.

2.3.2 En què es diferencia del Pokémon “normal”

El VGC es diferencia del mode Adventure del joc original en diversos aspectes fonamentals:

1. **Format de Dobles (2 vs 2)**: A diferència de les batalles Singles del mode Adventure, el VGC oficial és exclusivament de **Dobles**: cada jugador té dos Pokémon simultàniament al camp de batalla (Figura 2.5). Això multiplica l'espai de decisions i introdueix conceptes com la sinergia entre aliats, els atacs d'àrea, i la protecció (*Protect*).
2. **Team Preview i Bring 6, Pick 4**: Abans de cada partida, ambdós jugadors veuen els 6 Pokémon de l'equip rival i n'escullen 4 per competir. Aquesta fase de selecció és en si mateixa un joc d'informació imperfecta basat en la predicció.
3. **Decisió simultània**: Ambdós jugadors escullen les accions dels seus dos Pokémon al *mateix temps*, sense conèixer la decisió rival. Això obliga a preveure les accions de l'oponent.
4. **Regulació per temporades**: El VGC canvia les regles cada temporada i cada generació de jocs, redefinint quins Pokémon són legals i quines mecàniques s'apliquen. Això genera un metagame en constant evolució.
5. **Restriccions (Clauses)**: La *Species Clause* (no repetir espècies) i la *Item Clause* (no repetir objectes) forcen diversitat estratègica als equips.



Figura 2.5: Captura d'un combat de Dobles VGC al simulador Pokémon Showdown. S'observen els quatre Pokémon al camp (dos per bàndol), les barres de vida, les condicions actives i l'efecte climàtic.

2.3.3 El Metagame i les Generacions

El terme **metagame** (“meta”) fa referència a les estratègies dominants en un moment donat. Cada nova generació de jocs de Pokémon (de la Generació I a la IX actual) introdueix noves espècies, moviments, objectes i mecàniques que transformen radicalment les estratègies viables.

Per exemple, la *Generació VIII* (Pokémon Sword/Shield) va introduir el *Dynamax*, una mecànica que permetia duplicar la vida d'un Pokémon temporalment, mentre que la *Generació IX* (Pokémon Scarlet/Violet) va substituir-la per la *Teracristal·lització*, que permet canviar el tipus d'un Pokémon en ple combat. Cadascuna generació presenta un entorn de complexitat diferent amb el qual l'agent d'IA hauria de poder operar.

Comunitats com *Smogon* [23] mantenen una classificació organitzada de Pokémon per tiers d'ús (OU, UU, RU, etc.) i compilen guies estratègiques que defineixen el meta de cada generació i format, sent un recurs valuós tant per a jugadors humans com per a investigadors.

2.4 L'Entorn Virtual: Eines de Simulació

La naturalesa tancada dels jocs de consola originals impedeix la manipulació directa de les batalles per part d'un agent d'IA. Per això, la comunitat ha desenvolupat eines de codi obert que habiliten la investigació. Existeixen diverses alternatives — com *Pokémon Essentials* (motor RPG genèric) o simuladors ad-hoc—, però cap d'elles ofereix la combinació de precisió mecànica, validació comunitària i escalabilitat necessàries per a recerca rigorosa. Dues eines fonamentals per a aquest projecte destaquen sobre la resta:

2.4.1 Pokémon Showdown

Pokémon Showdown [13] és el simulador de batalles competitives més utilitzat del món. Desenvolupat en JavaScript/TypeScript, replica amb precisió mil·limètrica totes les mecàniques del joc oficial (fórmules de dany, taula de tipus, IVs/EVs, condicions d'estat, etc.) mitjançant enginyeria inversa dels jocs originals. L'elecció de Showdown com a motor de simulació respon a tres factors:

1. **Precisió acreditada:** dècades de validació competitiva per part de la comunitat de Smogon [23] garanteixen la fidelitat del motor de càlcul.
2. **Escalabilitat horitzontal:** permet llançar múltiples instàncies de servidor independents, habilitant paral·lelisme massiu per a benchmarking estadísticament significatiu.
3. **Mode headless:** pot funcionar com a **servidor local** sense interfície gràfica, permetent executar milers de batalles per minut de forma automatitzada.

Aquesta combinació el converteix en l'entorn de simulació ideal per entrenar i avaluar agents d'IA a gran escala.

2.4.2 Poke-env

Poke-env [17] és una llibreria de Python que actua com a capa d'abstracció (*API wrapper*) sobre Pokémon Showdown. Tradueix el flux de comunicació del servidor (missatges WebSocket) a objectes Python estructurats, proporcionant:

- Representació de l'estat de la batalla com a objectes accessibles (Pokémon al camp, vida restant, moviments disponibles, etc.).
- Interfície estàndard compatible amb l'estructura d'*OpenAI Gym* [2] per definir agents de *Reinforcement Learning*.
- Agents de referència (*baselines*) integrats per a validació i comparació.

La combinació de Showdown com a motor i Poke-env com a interfície estableix l'ecosistema complet que habilita aquest TFM: un entorn reproduïble, escalable i fidel a les regles oficials del joc. La seva configuració tècnica detallada es presenta al Capítol 3.

2.5 Intel·ligència Artificial i Tècniques Computacionals

Aquesta secció presenta les tècniques algorísmiques i d'aprenentatge automàtic rellevants per al projecte, des dels mètodes clàssics de cerca fins a les arquitectures neuronals modernes.

2.5.1 Algorismes de Cerca Clàssics

Minimax amb Esporgada Alfa-Beta. L'algorisme *Minimax* [10] modela un joc de dos jugadors com un arbre de decisions on un jugador (*Max*) intenta maximitzar la seva puntuació i l'altre (*Min*) intenta minimitzar-la. Explora totes les branques possibles fins a una profunditat determinada, assignant valors als estats terminal i propagant-los cap enrere. L'**esporgada alfa-beta** redueix el nombre de nodes explorats descartant branques que no poden millorar el resultat ja trobat.

Limitació: El Minimax funciona bé en jocs amb un factor de ramificació moderat (escacs: ~ 35), però col·lapsa en entorns estocàstics d'alta dimensionalitat com Pokémon, on la combinació de variables ocultes i aleatorietat fa inviable l'exploració exhaustiva.

Cerca d'Arbre de Monte Carlo (MCTS). L'MCTS [5] supera les limitacions del Minimax emprant simulacions aleatòries per estimar el valor d'un estat. El procés iteratiu segueix quatre fases:

1. **Selecció:** Des de l'arrel, selecciona el node fill més prometedor (balançant exploració i explotació amb UCB1).
2. **Expansió:** Afegeix un nou node fill al node seleccionat.

3. **Simulació (*Rollout*)**: Des del nou node, es juga una partida aleatòria fins al final.
4. **Retropropagació**: El resultat de la simulació s'actualitza cap amunt a tots els nodes predecessors.

Repetint milers de vegades aquest cicle, l'MCTS convergeix cap a una estimació robusta de quina acció és millor, sense necessitat d'explorar tot l'arbre. Va ser clau en l'èxit d'AlphaGo [20].

2.5.2 Xarxes Neuronals Artificials

Les xarxes neuronals [11] són funcions d'aproximació universals compostes per capes de neurones interconnectades. Cada neurona aplica una transformació lineal seguida d'una funció d'activació no lineal. Mitjançant l'*aprenentatge* (ajust dels pesos per *backpropagation* sobre dades), una xarxa pot aprendre a reconèixer patrons complexos.

Les arquitectures més rellevants per a jocs inclouen:

- **Xarxes Denses (MLP)**: Capes totalment connectades, adequades per processar vectors d'estat de dimensió fixa.
- **Xarxes Recurrents (LSTM/GRU)**: Incorporen memòria temporal, útils per capturar seqüències d'accions passades i deduir l'estat ocult en POMDPs.
- **Xarxes Convolucionals (CNN)**: Especialitzades en dades espacials, menys rellevants en el nostre cas.

2.5.3 Reinforcement Learning (RL)

El **Reinforcement Learning** (Aprenentatge per Reforç) [24] és un paradigma d'aprenentatge automàtic on un **agent** interactua amb un **entorn** de forma iterativa: observa l'estat, executa una acció, rep una recompensa, i repeteix. L'objectiu és aprendre una política π que maximitzi la recompensa acumulada a llarg termini.

El RL es diferencia de l'aprenentatge supervisat en un aspecte crucial: l'agent **no disposa d'exemples correctes** (*labels*), sinó que ha de **descobrir** quines accions són bones mitjançant l'experiència pròpia (prova i error).

Els mètodes de RL es classifiquen en:

- **Value-based** (basat en valor): L'agent aprèn una funció $Q(s, a)$ que estima el valor de fer una acció a en l'estat s . Exemple: DQN.
- **Policy-based** (basat en política): L'agent aprèn directament la política $\pi(a | s)$ com a distribució de probabilitat sobre les accions.
- **Actor-Critic**: Combina ambdós: una xarxa *Actor* aprèn la política i una xarxa *Critic* avalua com de bona és cada acció. Exemple: PPO.

DQN (Deep Q-Network)

Introduït per DeepMind [14], DQN aproxima la funció $Q(s, a)$ amb una xarxa neuronal profunda. Dues innovacions clau:

- **Experience Replay**: Emmagatzema les experiències (s, a, r, s') en un buffer i entrena sobre mostres aleatòries, trencant la correlació temporal.
- **Target Network**: Utilitza una còpia fixa de la xarxa Q per calcular els objectius d'entrenament, estabilitzant l'aprenentatge.

DQN funciona bé amb espais d'acció discrets i moderats, com és el cas de les batalles Pokémon (on les accions legals per torn són un conjunt finit d'atacs i substitucions).

PPO (Proximal Policy Optimization)

PPO [18] és un algorisme Actor-Critic desenvolupat per OpenAI que ha demostrat un rendiment molt robust en entorns complexos. Intenta resoldre el problema central del RL basat en política: actualitzacions massa grans que destrueixen l'aprenentatge acumulat. Ho aconsegueix amb la tècnica de *Clipping*: restringeix la magnitud de cada actualització de la política dins d'un rang $[1 - \epsilon, 1 + \epsilon]$, evitant canvis bruscos. Formalment, l'objectiu de PPO és:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.3)$$

on $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ és la ràtio de probabilitat entre la política nova i l'antiga, i $\hat{A}_t$ és l'estimació de l'avantatge.

Aquesta estabilitat és particularment valuosa en entorns **altament estocàstics com Pokémon**, on una recompensa enganyosament alta (p. ex., guanyar per un cop crític afortunat) podria desestabilitzar completament l'entrenament sense la protecció del clipping. A més, PPO permet entrenar contra múltiples oponents amb estratègies heterogènies sense perdre estabilitat, una propietat essencial per al currículum progressiu d'oponents que s'implementa en aquest TFM.

Action Masking: Gestió d'Accions Il·legals

En dominis com Pokémon, no totes les accions són legals en cada estat. A cada torn, la disponibilitat d'accions varia dinàmicament: un moviment sense PP no es pot usar, un Pokémon debilitat no pot ser seleccionat per al canvi, i certes mecàniques (com la Teracristal·lització) només es poden activar un cop per batalla. Forçar l'agent a explorar accions il·legals durant l'entrenament és ineficient i pot desestabilitzar la política.

La tècnica d'**action masking** [huang'action'masking] resol aquest problema restringint l'espai d'accions disponibles a cada pas temporal. En la pràctica, s'implementa multiplicant els *logits* de sortida de la xarxa de política per una màscara binària $m_t \in \{0, 1\}^{|A|}$ abans d'aplicar la funció *softmax*:

$$\pi_\theta(a | s, m) = \frac{e^{z_a} \cdot m_a}{\sum_{a'} e^{z_{a'}} \cdot m_{a'}} \quad (2.4)$$

on z_a són els logits i $m_a = 1$ si l'acció a és legal, 0 altrament. Això garanteix $\pi(a_{il\cdot legal} | s) = 0$ sense necessitat de penalitzar l'agent posteriorment.

MaskablePPO [16] és una extensió de PPO que incorpora nativament aquest mecanisme, millorant significativament l'eficiència i convergència en entorns amb espais d'acció dinàmics. Per a Pokémon, la màscara es genera a cada torn a partir de la informació del servidor: moviments amb PP > 0, Pokémon de reserva no debilitats, i disponibilitat de mecàniques especials.

Reward Shaping: Recompenses Intermèdies

Les batalles Pokémon proporcionen realimentació inherentment escassa: la recompensa principal (+1 per victòria, -1 per derrota) només es rep al final de la batalla, que pot durar desenes de torns. Sense orientació intermèdia, l'agent explora aleatòriament sense senyal clara de millora, dificultant enormement la convergència.

El **Reward Shaping** [ng'reward'shaping] és la tècnica de dissenyar recompenses intermèdies que accelerin l'aprenentatge sense alterar la política òptima. Ng et al. van demostrar que, sota certes condicions (funcions de potencial), les recompenses modelades preserven l'optimalitat de la política original. En el context de Pokémon, exemples de senyals intermèdies inclouen:

- Recompensa positiva per cada Pokémon rival debilitat (incentiva el progrés ofensiu).

- Penalització per torn transcorregut (desincentiva l'*stalling* i fomenta decisions ràpides).
- Bonificació per ús de moviments superefectius (ensenya el domini de la Taula de Tipus).

El calibratge d'aquestes senyals és crític: recompenses intermèdies massa fortes poden ensenyar comportaments subòptims (p. ex., prioritzar dany immediat sobre el posicionament estratègic a llarg termini). El disseny concret de la funció de recompensa s'exposa al Capítol 5.

Curriculum Learning: Dificultat Progressiva

L'**aprenentatge per currículum** [bengio'curriculum] és una estratègia d'entrenament on l'agent s'exposa progressivament a oponents o tasques de dificultat creixent, anàloga a com un jugador humà progressa des de partides casuais fins a competicions d'alt nivell.

Per a Pokémon, el currículum es materialitza en fases seqüencials d'entrenament:

1. **Fase inicial (*warm-up*):** Entrenament contra oponents aleatoris sense estratègia, permetent a l'agent descobrir accions bàsiques.
2. **Fase intermèdia:** Oponents heurístics simples (p. ex., seleccionar l'atac de major potència).
3. **Fase avançada:** Oponents heurístics complexos amb canvis estratègics, predicció de tipus i gestió de velocitat.

Aquesta progressió evita dos problemes comuns en RL: (1) l'**oblit catastròfic**, on entrenar contra nous oponents esborra el coneixement previ; i (2) la **convergència prematura**, on l'agent s'especialitza excessivament contra un oponent dèbil i fracassa davant rivals reals amb estratègies diverses.

2.5.4 Models de Llenguatge de Gran Escala (LLMs)

Els **LLMs** (*Large Language Models*) són xarxes neuronals de tipus *transformer* entrenades sobre corpus massius de text que exhibeixen capacitats emergents de raonament, generació de codi i anàlisi estratègica [30]. La seva capacitat per processar descripcions textuais complexes i realitzar inferències multi-pas sense etiquetatge explícit els fa atractius per a dominis amb informació parcial on la formalització completa és difícil.

Pokechamp i PokeLLMon: Aplicació a Pokémon

Pokechamp [12] (*PokéChamp: an Expert-level Minimax Language Agent*, ICML 2025) és un repositori de codi obert que combina dues tècniques presentades anteriorment — Minimax i Models de Llenguatge de Gran Escala (LLMs) — per crear un agent de nivell expert en batalles Pokémon. Pokechamp utilitza un LLM per raonar sobre l'estat de la batalla (avaluar posicions, predir moviments del rival) i combina aquest raonament amb una cerca Minimax d'un pas per seleccionar l'acció òptima.

El repositori inclou diversos agents amb aproximacions diferents:

- **Pokechamp (LLM + Minimax):** L'agent principal. Utilitza un LLM per generar avaluacions heurístiques dels estats i una cerca Minimax guiada per optimitzar la decisió final.
- **PokeLLMon (LLM):** Un agent alternatiu basat exclusivament en LLM, que pren decisions directament a partir de la descripció textual de la batalla sense cerca addicional.
- **Abyssal (Heurístic):** Agent basat en regles que avalua matchups de tipus i estima el dany.

- **OneStep (Heurístic):** Agent amb estimació de dany a un torn vista (*one-step lookahead*).

En el context de Pokémon, els LLMs permeten a l'agent "llegir" l'estat de la batalla en format textual i "raonar" sobre quina acció prendria un jugador expert, processant informació parcial i fent inferències sobre les intencions del rival. Això els converteix en un mètode prometedor per abordar POMDPs on la descripció de l'estat és naturalment lingüística.

En aquest TFM, els agents de Pokechamp no constitueixen una contribució pròpia sinó que es fan servir com a *baselines* externs — tant heurístics com basats en LLM — contra els quals comparar el rendiment dels agents desenvolupats, complementant els agents nadius de poke-env. Els detalls d'integració i configuració dels LLMs s'exposen al Capítol 5.

2.6 Estat de l'Art: Intel·ligència Artificial Aplicada a Pokémon

Malgrat que Pokémon és un domini relativament jove en la recerca d'IA acadèmica comparat amb escacs o Go, existeix un corpus creixent de treballs que han adaptat tècniques clàssiques i modernes al context de batalles competitives. Aquesta secció revisa els treballs més rellevants i posiciona la contribució del present TFM.

2.6.1 Investigació Inicial: Poke-env i Agents Heurístics

El punt d'inflexió per a la recerca en IA aplicada a Pokémon va ser el desenvolupament de **poke-env** [pokemon'ai'sahovic, 17] per Haris Sahovic (2020). Aquesta llibreria va establir una interfície Python estandarditzada sobre Pokémon Showdown, compatible amb OpenAI Gym [2], que va permetre per primera vegada aplicar algorismes de RL de forma directa al domini.

Els primers agents desenvolupats amb poke-env eren majoritàriament heurístics: calculaven el dany esperat per cada moviment, aplicaven regles de *switching* basades en la Taula de Tipus, i seleccionaven l'acció amb la puntuació més alta. Aquests agents, tot i la seva simplicitat, van establir les primeres línies base (*baselines*) contra les quals mesurar el progrés. La pròpia llibreria incorpora tres agents de referència — *RandomPlayer*, *MaxBasePowerPlayer* i *SimpleHeuristicPlayer* — que representen nivells creixents de sofisticació heurística.

Paral·lelament, la comunitat de **Smogon University** [23] ha mantingut durant anys un corpus massiu d'anàlisi estratègica manual sobre el metagame competitiu: catàlegs de Pokémon per *tiers* d'ús, guies de matchups, i calculadores de dany. Aquesta base de coneixement expert ha informat l'arquitectura de múltiples agents heurístics en la literatura.

2.6.2 Reinforcement Learning i Xarxes Neuronals

Huang et al. (2023) [pokemon'rl'huang] van publicar "*Action Q-Transformer: Visual Explanation in Deep Reinforcement Learning with Encoder-Decoder Model for Pokémon Battle*" a la IEEE Conference on Games. El seu treball entrena agents DQN sobre Pokémon Showdown utilitzant una arquitectura *encoder-decoder* amb mecanisme d'atenció, aconseguint superar *baselines* heurístics simples. A més, incorporen explicabilitat visual per interpretar les decisions de l'agent.

Altres treballs dins la comunitat han explorat aproximacions complementàries:

- **PPO amb vectorització d'estat:** Discretitzar l'estat de batalla en vectors densos i entrenar PPO, una aproximació similar a la d'aquest TFM però generalment limitada a formats *Singles* i sense *action masking* explícit.
- **LSTM per capturar historial:** Utilitzar xarxes recurrents per modelar dependències temporals, intentant inferir l'estat ocult en POMDPs a partir de la seqüència d'observacions passades.

- **Self-play:** Entrenar agents competint contra si mateixos en iteracions progressives, seguint l'aproximació d'AlphaZero [19] adaptada al domini de Pokémon.

No obstant, cap d'aquests treballs ha abordat de forma sistemàtica el format de Dobles (2 vs 2), que multiplica exponencialment l'espai d'accions i requereix coordinació multi-agent.

2.6.3 Models de Llenguatge: Pokechamp i PokeLLMon

El treball més recent i impactant és **Pokechamp** [12] (Lee et al., ICML 2025), que representa un salt qualitatiu en la sofisticació dels agents. Pokechamp demostra que els LLMs posseeixen raonament estratègic suficient per jugar a nivell d'experts humans quan es combinen amb cerca Minimax d'un pas. Els seus resultats suggereixen que els agents basats en LLM superen en molts casos els agents purament heurístics en el format de Singles (Gen 9).

Tanmateix, els agents LLM presenten limitacions importants: (1) la latència d'inferència dificulta l'entrenament iteratiu a gran escala; (2) el cost computacional dels LLMs és ordres de magnitud superior al d'un forward pass d'una xarxa MLP; i (3) la seva dependència de descripcions textuais pot introduir biaixos no controlats.

2.6.4 Connexió amb Altres Dominis d'Informació Imperfecta

La investigació en jocs amb informació imperfecta té una tradició consolidada en IA que informa indirectament aquest treball:

- **Pòquer** [3, 4]: Sistemes com Libratus i Pluribus han assolit nivell superhumà en *No-Limit Texas Hold'em* utilitzant *counterfactual regret minimization* i abstraccions de l'arbre de joc. Les tècniques de gestió d'incertesa d'aquests sistemes són parcialment transferibles a Pokémon.
- **StarCraft II** [28]: AlphaStar integra RL multi-agent amb *self-play* per dominar un entorn d'informació imperfecta en temps real. Els mecanismes de mostreig d'oponents durant l'entrenament han inspirat aproximacions similars en jocs per torns.

2.6.5 Buit de la Recerca

De l'anàlisi de l'estat de l'art s'identifiquen els següents buits que aquest TFM es proposa abordar:

1. **Integració sistemàtica de paradigmes:** No existeix un marc unificat que compari heurístiques, RL i agents LLM en una matriu de benchmarking comuna amb milers de batalles per parella.
2. **Format de Dobles (2 vs 2):** La majoria de recerca s'ha centrat en Singles. El format Dobles introdueix coordinació multi-agent, atacs d'àrea i mecàniques de protecció que requereixen arquitectures específiques.
3. **Action masking explícit:** Pocs treballs incorporen MaskablePPO per gestionar espais d'acció dinàmics i variables en el temps de forma nativa.
4. **Validació cross-generacional:** Demostrar que els agents generalitzen entre generacions de joc (Gen 8 amb Dynamax vs. Gen 9 amb Teracristallització), cadascuna amb mecàniques i metagames diferents.

2.7 Justificació i Posicionament de la Recerca

2.7.1 Per què Pokémon és un Cas d'Estudi Ideal

Pokémon combina de forma natural totes les característiques que defineixen un entorn desafiant per a la IA, convertint-lo en un banc de proves particularment ric:

- **Estocasticitat alta:** el dany varia aleatòriament entre el 85% i el 100%, els atacs poden fallar per precisió, i els cops crítics (+50% dany) ocorren amb un $\approx 4\%$ de probabilitat. Cap estratègia garanteix un resultat al 100%.
- **Informació imperfecta (POMDP):** els atacs, estadístiques ocultes (EVs, IVs, Nature), objectes equipats i estratègia del rival no es revelen fins que s'observen empíricament durant la partida.
- **Decisions simultànies:** en el format competitiu, ambdós jugadors decideixen les seves accions al **mateix temps**, sense conèixer la decisió rival. Això obliga a la predicció de l'adversari.
- **Espai d'accions combinatori:** especialment en el format de Dobles (2 vs 2), on cal coordinar les accions de dues unitats pròpies contra dues rivals (≤ 81 combinacions per torn).
- **Joc de suma zero [29]:** el guany d'un jugador és la pèrdua exacta de l'altre, sense possibilitat d'equilibris cooperatius.
- **Ecosistema obert:** existeixen simuladors de codi obert (Pokémon Showdown) i llibreries d'IA (poke-env, Pokechamp) que habiliten la investigació reproducible a gran escala.

2.7.2 Pokémon com a Laboratori d'IA

El propòsit d'investigar el competitiu de Pokémon no és crear un bot imbatible per entreteniment, sinó utilitzar aquest ecosistema com un **laboratori controlat** per validar les capacitats i limitacions d'arquitectures d'IA autònoma en escenaris realistes.

Pokémon VGC sintetitza reptes representatius de problemes reals d'enginyeria i gestió on un agent ha d'operar sota:

- **Soroll estocàstic alt:** les decisions òptimes poden resultar en fracàs per atzar, definint una taxa irreductible de pèrdues que cap model pot eliminar.
- **Informació parcial:** l'agent ha de mantenir un *belief state* implícit i deduir informació oculta a partir d'observacions acumulades al llarg de la batalla.
- **Adversaris intel·ligents i adaptatius:** l'oponent (humà o agent) modifica la seva estratègia dinàmicament en resposta a les accions observades.
- **Restriccions dinàmiques d'acció:** no totes les accions són legals en cada moment, requerint mecanismes d'*action masking* per a un entrenament eficient.

Construir agents que evolucionin des d'heurístiques preprogramades fins a models d'aprenentatge autònom permet validar la robustesa dels sistemes de decisió en entorns complexos. La disponibilitat d'eines obertes i la possibilitat de realitzar milers d'experiments controlats amb reproductibilitat total fan de Pokémon una plataforma excepcionalment adequada per a la recerca acadèmica en IA.

2.8 Síntesi i Pont vers la Metodologia

Aquest capítol ha establert el marc formal i contextual en què se situa el TFM. S’han recorregut: (1) l’evolució de la IA en jocs des de la informació perfecta fins als POMDPs complexos; (2) les mecàniques específiques de Pokémon que defineixen els espais d’estat i d’acció; (3) el format competitiu VGC amb decisions simultànies i coordinació multi-agent; (4) les eines de simulació que permeten la investigació reproducible; (5) les tècniques d’IA rellevants, amb èmfasi en PPO, *action masking*, *reward shaping* i *curriculum learning*; (6) l’estat de l’art en IA aplicada a Pokémon; i (7) la justificació i el posicionament de la recerca.

En particular, la naturalesa POMDP i altament estocàstica del combat Pokémon motiva l’ús d’arquitectures capaces d’operar sobre observacions parcials (vectorització de l’estat en 238 dimensions) i d’algorismes robustos com MaskablePPO que gestionen espais d’acció dinàmics. El *reward shaping* és essencial per superar el problema de recompenses escasses, i l’aprenentatge per currículum assegura una convergència estable contra oponents de complexitat creixent.

Sobre aquesta base teòrica s’articula la metodologia experimental descrita al Capítol 3, on es formalitzen els espais d’estats i accions, es defineixen les mètriques d’avaluació (Win-Rate com a mesura primària, sistema Elo [**elo·rating**] per a rànquings globals), i s’estructura el disseny experimental. El Capítol 5 tradueix posteriorment aquests conceptes en codi executable.

Capítol 3

Metodologia: Disseny Experimental

Per poder dissenyar, avaluar i entrenar agents d'IA en el metagame Pokémon presentat al capítol anterior, cal estructurar les bases experimentals del projecte. Aquest capítol detalla l'entorn de simulació, la complexitat del problema, els espais formals de treball i les mètriques d'avaluació. Finalment, s'exposa l'estratègia de desenvolupament en fases progressives.

3.1 Entorn de Simulació

3.1.1 Arquitectura Client-Servidor de Pokémon Showdown

Pokémon Showdown [13] és un simulador de combats Pokémon de codi obert que implementa la totalitat de les regles oficials de les generacions 1 a 9. A nivell tècnic, s'estructura com una aplicació **Node.js** que exposa un servidor de combats accessible via el protocol **WebSocket** (RFC 6455). Aquesta arquitectura client-servidor és fonamental per entendre com els agents d'IA interactuen amb el joc.

Llançament del Servidor Local

En un entorn de recerca, Showdown s'executa localment sense interfície gràfica ni autenticació:

```
node pokemon-showdown start --port 8000 --no-security
```

El flag `--no-security` desactiva el sistema d'autenticació (passwords, tokens), permetent que qualsevol client local es connecti directament. Cada instància del servidor ocupa un **port TCP exclusiu** i funciona com un procés Node.js independent que manté en memòria volàtil (RAM) l'estat de totes les batalles actives. No hi ha persistència a disc: quan el procés es tanca, tot l'estat es perd.

Protocol de Comunicació WebSocket

La comunicació entre un agent (client) i el servidor utilitza **WebSocket**, un protocol full-duplex sobre TCP que manté una connexió persistent. A diferència de HTTP (request-response), WebSocket permet que el servidor envii missatges al client en qualsevol moment sense necessitat de polling.

Els missatges de Showdown utilitzen un **format de text pla** amb una sintaxi basada en delimitadors `|`. Cada línia representa un event del combat:

```
|move|p1a: Pikachu|Thunderbolt|p2a: Gyarados  
|-damage|p2a: Gyarados|23/100  
|-supereffective|p2a: Gyarados  
|turn|5
```

L'estructura general és `|tipus_event|argument1|argument2|...`. Els prefixos `|-` denoten events secundaris (dany, efectes, estats alterats). Aquesta codificació en text pla és molt lleugera (~50–200 bytes per torn) comparada amb un format binari o JSON complet.

El Missatge request: El JSON d'Estat

El missatge més important per a un agent és el `|request|`, que el servidor envia a cada jugador al començament de cada torn. Conté un objecte JSON complet amb tota la informació disponible per prendre una decisió:

```
|request|{"active":[{"moves":[
  {"move":"Thunderbolt","id":"thunderbolt","pp":24,
   "maxpp":24,"target":"normal","disabled":false},
  {"move":"Iron Tail","id":"irontail","pp":24,
   "maxpp":24,"target":"normal","disabled":false},
  ...
]},{"side":{"pokemon":[
  {"ident":"p1: Pikachu","details":"Pikachu, L84, M",
   "condition":"187/187","active":true,
   "stats":{"atk":137,"def":98,"spa":148,"spd":118,"spe":178},
   "moves":["thunderbolt","irontail","surf","voltswitch"],
   "item":"lightball","ability":"Static"},
  ...
]}}
```

Aquest JSON conté: els moviments disponibles (amb PP restants, si estan desactivats), les estadístiques exactes dels Pokémon propis, l'objecte equipat, l'habilitat, i la condició actual (HP_actual/HP_màxim status). Notar que les estadístiques i objectes de l'oponent **no s'inclouen** mai — això formalitza la informació imperfecta.

Resposta de l'Agent: Ordres de Combat

L'agent respon amb una cadena de text simple que codifica la seva decisió:

- `/choose move 1` — Usar el primer moviment de la llista.
- `/choose move 3 -2` — Usar el tercer moviment apuntant al segon oponent (format Doubles).
- `/choose switch 4` — Substituir pel quart Pokémon de l'equip.
- `/choose move 1 terastallize` — Atacar amb Teracristallització activa.

Un cop el servidor rep les ordres d'ambdós jugadors, **resol el torn de forma atòmica**: aplica les regles de prioritat, compara velocitats, calcula dany amb la fórmula oficial (incloent un factor aleatori entre 0.85 i 1.0), i envia el flux de events resultant a ambdós clients.

Cicle de Vida Complet d'una Batalla

La Figura 3.1 il·lustra el cicle complet d'una batalla des de la connexió fins al resultat final.

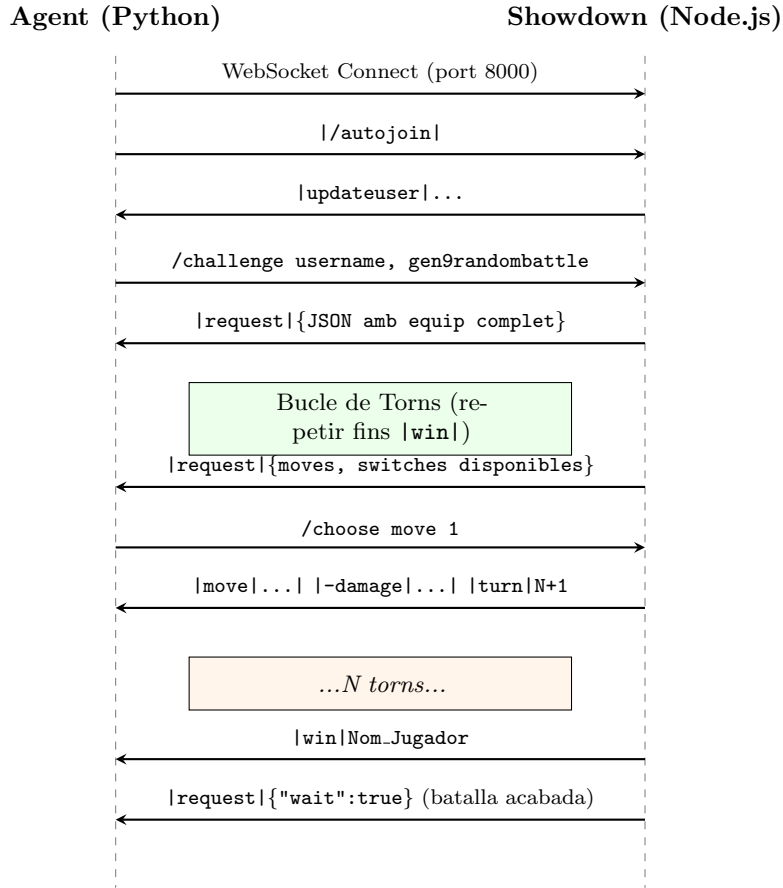


Figura 3.1: Diagrama de seqüència del protocol WebSocket entre un agent i Pokémon Showdown durant una batalla completa.

Punts importants d'aquest cicle:

- Tot l'estat del combat viu exclusivament a la **RAM del procés Node.js**. No hi ha cap escriptura a disc durant la batalla.
- El servidor és **completament determinista** donats els mateixos inputs i la mateixa llavor de RNG. Això permet la reproductibilitat dels experiments.
- La latència d'un torn (enviar ordre → rebre resultat) és típicament <5 ms en local, permetent velocitats de centenars de batalles per minut.
- Cada instància del servidor pot gestionar **múltiples batalles simultànies** de forma concurrent gràcies a l'event loop de Node.js.

3.1.2 Poke-env: La Capa d'Abstracció Python

La llibreria poke-env [17] actua com a pont entre el protocol WebSocket de Showdown i el codi Python dels agents. Proporciona una interfície orientada a objectes que permet als investigadors centrar-se en la lògica de decisió sense gestionar connexions de xarxa ni parsejar missatges de text.

Arquitectura Interna de Poke-env

La Figura 3.2 mostra com poke-env transforma els missatges crus del servidor en objectes Python estructurats.

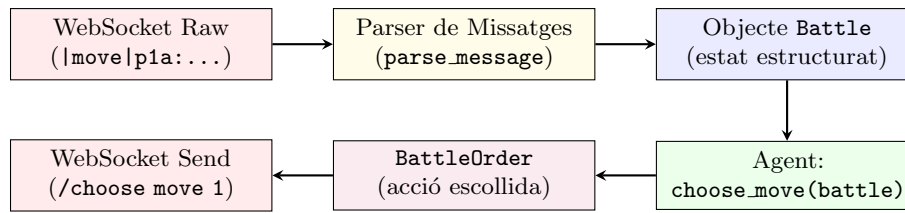


Figura 3.2: Pipeline intern de poke-env: del missatge WebSocket cru fins a la resposta de l'agent.

Classes Principals

- **Player:** Classe base abstracta per a qualsevol agent. El desenvolupador només necessita implementar el mètode `choose_move(battle)` que rep l'estat actual i retorna una acció. Poke-env s'encarrega de tota la gestió de connexió, autenticació i sincronització.
- **Battle:** Encapsula l'estat complet observable d'una batalla a cada torn. Conté: `active_pokemon`, `opponent_active_pokemon`, `available_moves`, `available_switches`, `weather`, `fields`, `side_conditions`, etc.
- **Pokemon:** Objecte amb atributs com `types`, `stats`, `boosts`, `status`, `current_hp_fraction`, `moves` (revelats), etc.
- **Move:** Conté `base_power`, `type`, `accuracy`, `category` (Physical/Special/Status), `priority`, i efectes secundaris.

Model de Concurrencia: asincio

Poke-env utilitza el framework `asyncio` de Python per gestionar múltiples batalles simultàniament dins d'un sol fil d'execució. El paràmetre `max_concurrent_battles` controla quantes batalles pot mantenir obertes un agent alhora (típicament 10–20). Internament, un `Semaphore` limita la creació de batalles noves fins que les anteriors finalitzin.

Quan es crida `player.battle_against(opponent, n_battles=100)`, poke-env:

1. Obre connexions WebSocket per ambdós jugadors.
2. Llança el challenge i espera l'acceptació.
3. Per cada torn, crida `choose_move()` i envia la resposta.
4. Un cop rep `|win|`, marca la batalla com a finalitzada i allibera el semàfor.

Tot aquest procés és **no-bloquejant**: mentre una batalla espera la resposta del servidor, altres batalles poden estar enviant les seves ordres.

Compatibilitat amb OpenAI Gym

Poke-env també exposa una interfície compatible amb l'estàndard *Gymnasium* [2], permetent l'ús directe d'algorismes de Reinforcement Learning (DQN, PPO, A2C). Aquesta interfície transforma cada torn en una tupla (`observation`, `reward`, `done`, `info`) i gestiona automàticament el cicle de vida de l'entorn.

3.1.3 Discrepàncies Tècniques: Poke-env vs Pokechamp

Durant el desenvolupament, s'ha identificat una dicotomia metodològica entre la versió oficial de *poke-env* i la versió utilitzada pel repositori *Pokechamp*. Aquesta distinció és crítica per a la reproductibilitat:

- **Compatibilitat d'Atributs:** La versió de *Pokechamp* utilitza una branca modificada de la llibreria que accedeix a atributs de batalla (com `active_pokemon`) de forma diferent a la versió 0.11+ oficial. Això ha obligat a crear una capa d'abstracció a `AgentFactory` per garantir que els agents d'ambdues procedències puguin coexistir en el mateix benchmark.
- **Seguiment Estricte (Strict Tracking):** Mentre que *Pokechamp* ignora certs errors de protocol de Showdown, *poke-env* oficial llança excepcions quan detecta moviments inesperats. La decisió metodològica ha estat relaxar aquest seguiment (`strict_battle_tracking=False`) per permetre la compatibilitat amb la lògica de tercers i assegurar la continuïtat de les simulacions de llarga durada.

3.1.4 Variables d'Estat Disponibles

L'objecte `Battle` que rep l'agent cada torn exposa les següents variables observables:

Variable	Descripció
HP propis	Percentatge i valor absolut de vida de cada Pokémon propi (actius i reserva).
HP rivals	Percentatge de vida dels Pokémon rivals observats al camp.
Moviments propis	Llista de fins a 4 moviments per Pokémon: nom, tipus, potència, precisió, PP restants.
Moviments rivals coneguts	Moviments observats de l'oponent durant la batalla (informació parcial acumulativa).
Tipus de Pokémon	Tipus primari i secundari de cada Pokémon al camp.
Boosts	Modificadors d'estadístiques actius ($\uparrow/\downarrow$ en Atac, Defensa, Velocitat, etc.).
Condicions (Status)	Estats alterats actius: Burn, Paralysis, Sleep, Poison, Freeze.
Condicions de Camp	Efectes globals: clima (Sol, Pluja, etc.), terrenys, hazards (<i>Stealth Rock</i> , etc.).
Objectes	Objecte equipat dels Pokémon propis; l'objecte rival queda ocult fins que s'activa.
Habilitats	Habilitat passiva de cada Pokémon; la rival es pot deduir pel seu efecte.
Team Preview	Composició dels 6 Pokémon rivals (només espècies, sense stats ni objectes).

Taula 3.1: Variables d'estat observables per l'agent a cada torn a través de *poke-env*.

Informació oculta: En tot moment, l'agent **desconeix** les EVs, IVs, Nature, moviments no revelats, objecte i habilitat dels Pokémon rivals, així com l'ordre dels Pokémon que l'oponent manté a la reserva. Això confirma formalment que l'entorn és un POMDP.

3.1.5 Necessitat d'Infraestructura Paral·lela

L'execució de benchmarks estadísticament significatius (10.000 partides per parella d'agents) amb un sol servidor resulta inviable per dues limitacions inherents al sistema descrit anteriorment:

1. **Limitació de Node.js:** Cada instància de Showdown utilitza un únic fil d'execució (event loop). Tot i que pot gestionar múltiples batalles concurrentment, el throughput es satura al voltant de 20–30 batalles simultànies per la serialització de la resolució de torns.
2. **Acumulació de memòria:** El Garbage Collector de Node.js no allibera completament la memòria fragmentada de batalles finalitzades. Després de milers de batalles, el procés pot créixer de ~80 MB a >500 MB, degradant el rendiment i arriscant un *Out of Memory* (OOM).

Per superar-ho, la metodologia d'avaluació d'aquest TFM es basa en un patró **Master-Worker** amb múltiples instàncies de Showdown executant-se en ports paral·lels. Cada worker és un procés Python independent que es comunica amb el seu propi servidor Showdown dedicat, eliminant qualsevol contenció entre experiments. Els detalls d'implementació d'aquesta arquitectura es descriuen al Capítol 5 (Secció 5.2).

3.1.6 Disseny de la Matriu d'Avaluació (Benchmark Matrix)

Per tal de garantir una comparació justa i exhaustiva entre els diversos agents, s'ha implementat una metodologia d'avaluació **per matriu completa**. Un *run* del benchmark no avalua un sol agent, sinó que processa una matriu creuada de N agents contra M oponents, resultant en $N \times M$ matchups independents.

- **Recuperació automàtica (Resume):** El sistema comprova l'existència de dades prèvies a disc. Si d'un matchup de 10.000 partides ja se n'han completat 2.000 en una execució anterior, el benchmark només llançarà les 8.000 restants. Això fa el procés tolerant a interrupcions (talls de corrent, errors de xarxa, reinicis del sistema).
- **Consistència de Formats:** Tots els agents del benchmark s'avaluen sota exactament el mateix format i generació (p. ex. `gen9randombattle`), garantint que les diferències de rendiment es deuen exclusivament a la qualitat de la decisió, no a variacions de l'entorn.
- **Aleatorietat controlada:** En el format *Random Battle*, els equips són assignats aleatòriament pel servidor Showdown a cada partida. Amb 10.000 partides per matchup, la llei dels grans nombres garanteix que qualsevol avantatge puntual per equip es dilueix estadísticament.

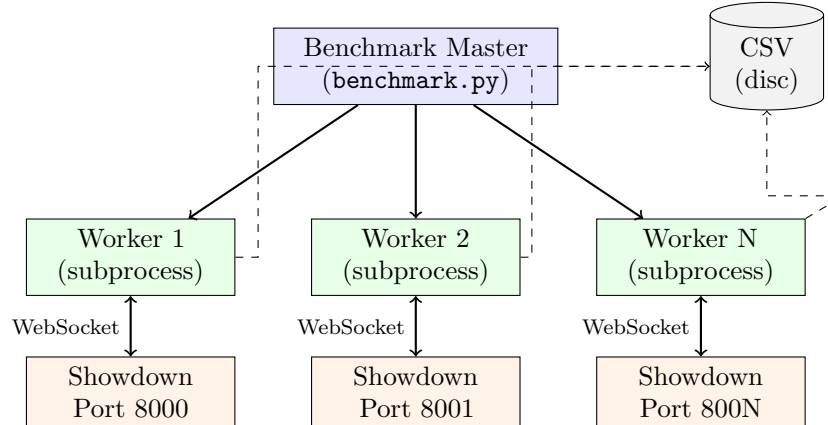


Figura 3.3: Arquitectura Master-Worker: cada worker és un procés Python aïllat que es comunica amb el seu propi servidor Showdown via WebSocket i escriu resultats parcials a disc.

3.2 Complexitat del Problema

3.2.1 Diferències entre Generacions

Cada generació de jocs de Pokémon defineix un conjunt diferent de regles, espècies disponibles i mecàniques especials. Això impacta directament en la complexitat del problema:

- **Nombre de Pokémon legals:** Varia entre ~ 150 (Generació I) i > 1.000 (Generació IX).
- **Mecàniques especials:** Mega-Evolucions (Gen VI–VII), Z-Moves (Gen VII), Dynamax (Gen VIII), Teracristal·lització (Gen IX) afegeixen accions extra i modificadors.

- **Objectes i habilitats:** La base creix a cada generació, augmentant les combinacions possibles.

Per gestionar aquesta variabilitat, l'arquitectura del projecte s'ha dissenyat per ser flexible: el format i la generació es passen com a paràmetres configurables, permetent entrenar i avaluar agents en qualsevol combinació.

3.2.2 Selecció de Generacions i Verificació de Formats

El projecte s'ha dissenyat per ser agnòstic a la generació, però l'avaluació real depèn de la disponibilitat de formats al servidor Showdown. Durant la fase d'experimentació, es va dur a terme una auditoria dels formats suportats:

- **Generacions 8 i 9:** Suporten nativament el format **Random Doubles Battle**, la qual cosa permet una avaluació directa de les heurístiques contra baselines estàndard.
- **Generació 7:** Tot i ser una generació d'alt interès competitiu, es va descobrir que el format oficial de *Random Doubles* no està habilitat per defecte en els servidors locals de Showdown (estant limitat a Singles).

Aquesta limitació tècnica va portar a la decisió metodològica de centrar el gruix de l'estudi en les ****Generacions 8 i 9****, on les dades són directament comparables amb l'ecosistema competitiu actual.

La diferència de complexitat entre formats és considerable:

Dimensió	Singles (1v1)	Doubles (2v2)
Pokémon al camp	1 propi, 1 rival	2 propis, 2 rivals
Accions per torn	≤ 9 (4 atacs + 5 switches)	$\leq 9 \times 9 = 81$ combinacions
Objectius d'atac	1 (el rival)	3 (rival 1, rival 2, aliat)
Sinergia	No aplica	Crític (proteccions, atacs spread, suport)
Decisions simultànies	2 (1 per jugador)	4 (2 per jugador)

Taula 3.2: Comparació de la complexitat dels formats Singles i Doubles.

El format de **Doubles** multiplica exponencialment l'espai de decisions: cada jugador ha de coordinar les accions de dos Pokémon simultàniament, tenint en compte sinergies, atacs d'àrea, proteccions i l'interacció entre els quatre Pokémon al camp. Per aquesta raó, el projecte aborda primer el format Singles com a prova de concepte i després escala al format Doubles.

3.3 Espai d'Estats i Espai d'Accions

Tot problema de RL ha d'estar encapsulat formalment per poder ser processat per un algorisme.

3.3.1 Espai d'Estats (S)

El conjunt S agrupa tota la informació observable per l'agent en un torn donat. Això constitueix un entorn parcialment observable (POMDP), on l'agent només accedeix a una observació $ODUMMY_UNDERSCOREt \in \mathbb{R}^d$ que resumeix:

- Vida (HP) restant de tots els Pokémon visibles (propis i rivals).
- Modificadors d'estadístiques actius (*boosts*).
- Condicions d'estat (status) actives.

- Informació de camp (clima, terreny, hazards).
- Historial acumulat de moviments i tipus observats del rival.

En el format de Singles, el vector d'observació resultant té de l'ordre de $d \sim 50\text{--}100$ dimensions. En Doubles, es duplica aproximadament per la presència de dos Pokémon actius per bàndol.

3.3.2 Espai d'Accions (A)

L'espai d'accions A conté els moviments i substitucions legals disponibles:

- **Singles:** Fins a 4 atacs + fins a 5 switches = màxim 9 accions discretes per torn.
- **Doubles:** L'agent ha d'escollir una acció per a *cadascun* dels dos Pokémon al camp. A més, cada atac ha d'especificar l'objectiu (*target*): rival 1, rival 2, o aliat. Això genera un producte cartesià de possibilitats que pot arribar a desenes de combinacions legals per torn.

Cal notar que l'espai d'accions és **dinàmic**: varia a cada torn segons els Pokémon vius, els PP restants, si un Pokémon ha estat derrotat i obliga un switch forçós, etc.

3.4 Mètriques d'Avaluació

La naturalesa estocàstica del combat Pokémon invalida qualsevol avaluació basada en poques partides: un sol resultat pot estar dominat per l'atzar. Per tant, totes les comparacions entre agents es realitzen sobre sèries massives de batalles (≥ 100 per parella).

3.4.1 Mètriques Primàries

- **Win-Rate (%)**: Percentatge de victòries de l'agent sobre el total de batalles disputades contra un oponent concret. És la **mètrica principal i definitiva** del projecte a l'hora de jutjar si un agent és superior a un altre; totes les altres mesures s'interpreten com a suport qualitatiu a aquesta.
 - $\sim 50\%$: L'agent no millora respecte un rival aleatori simètric.
 - $> 60\%$: Avantatge significatiu; l'agent demostra intel·ligència estratègica.
 - $> 85\%$: Domini clar; l'agent supera àmpliament l'oponent.
 - 100% : Intrínsecament impossible en un entorn estocàstic com Pokémon.

3.4.2 Mètriques Secundàries

Per complementar el Win-Rate i obtenir una visió més granular del comportament dels agents, es recullen mètriques addicionals:

- **Durada de les partides (torns)**: Un agent eficient tendeix a guanyar en menys torns. Partides molt llargues poden indicar indecisió o incapacitat d'aprofitar avantatges.
- **Pokémon derrotats (propis)**: Quantes unitats ha perdut l'agent durant la partida. Un nombre baix indica millor gestió defensiva.
- **Pokémon derrotats (rivals)**: Quantes unitats rivals ha eliminat l'agent. Pot diferenciar entre victòries ajustades i dominants.
- **Pokémon restants vius**: Tant propis com rivals al final de la partida. Indica el marge de victòria o derrota.

- **Vida total restant (%)**: Suma de la vida percentual de tots els Pokémon supervivents (propis i rivals). Aporta una mesura contínua i més granular que el simple recompte de Pokémon vius.

3.5 Disseny dels Experiments

Amb l'entorn de simulació i les mètriques definides, cal especificar com s'utilitzen per avaluar els diferents agents. Sense entrar encara en els resultats numèrics (Capítol 6), aquesta secció descriu l'esquema general del disseny experimental.

3.5.1 Tipus de Matchups

Els experiments es basen en sèries de batalles entre parelles d'agents (A, B) , on cada agent pot ser:

- Un agent heurístic propi (versions v1–v6 per Singles).
- Un agent de Reinforcement Learning entrenat amb el pipeline descrit al Capítol 5.
- Un baseline natiu de *poke-env* (RandomPlayer, MaxBasePowerPlayer, SimpleHeuristicPlayer).
- Un agent extern del repositori Pokechamp (Abyssal, SafeOneStep, Pokechamp, PokeLLMon), quan el format i generació són compatibles.

Per a cada parella (A, B) es defineixen **sèries de com a mínim 100 batalles**, amb equips inicials aleatoris dins del format triat (p. ex. `gen9randombattle` per a Singles). L'ordre de sortida (qui comença com a jugador 1 o 2) es balanceja perquè l'avantatge inicial no es barregi amb la qualitat de l'agent.

3.5.2 Formats i Condicions

Tot i que el projecte està dissenyat per donar suport a múltiples generacions i formats, els experiments d'aquest TFM se centren principalment en:

- **Singles (1 vs 1)** com a banc de proves principal per a heurístiques i RL.
- La generació i el format de Random Battles adequats per a la integració amb Pokechamp (Gen 9 Singles), quan s'inclouen aquests agents externs.

Les condicions d'entorn (clima, hazards, objectes, habilitats) són les definides pel simulador oficial de *Showdown* per al format escollit; el TFM no modifica les regles del joc, sinó que s'hi adapta.

3.5.3 Agregació de Resultats

Per a cada sèrie de batalles es calcula:

- El **Win-Rate** de cada agent, que és la mètrica principal utilitzada per comparar-los.
- Les mitjanes i distribucions de les mètriques secundàries descrites a la Secció 3.4, que es faran servir al Capítol 6 per interpretar qualitativament el comportament dels agents (eficiència temporal, agressivitat, solidesa defensiva, etc.).

Aquest disseny experimental garanteix que qualsevol diferència de rendiment observada en els resultats sigui estadísticament fiable i no deguda a la variabilitat inherent de poques partides.

3.5.4 Propera Fase: Anàlisi de la Distribució de Pokémon

Un dels reptes immediats un cop finalitzada l'avaluació massiva és l'anàlisi de les dades recollides. Tot i que els equips a *Random Battles* són generats aleatòriament pel servidor Showdown, aquests segueixen distribucions de probabilitat internes.

Està previst realitzar un estudi estadístic sobre el *usage-rate* i el *win-rate* individual de cada espècie de Pokémon dins d'aquests formats aleatoris. Aquesta anàlisi permetrà identificar si certs Pokémon tenen un impacte desproporcionat en la victòria independentment de l'agent que els controla, proporcionant una base per a una futura "normalització" dels resultats segons la qualitat de l'equip rebut.

3.6 Fases de Desenvolupament i Models de Referència

El desplegament d'agents sobre *Showdown* s'ha estructurat metodològicament en dues grans fases consecutives per garantir una monitorització i isolació de variables controlada.

3.6.1 Fase 1: Disseny de la Lògica de Decisió Heurística

Abans de procedir amb arquitectures d'aprenentatge automàtic, s'ha dissenyat un sistema d'heurístiques expertes. L'objectiu no és només crear un agent fort, sinó **formalitzar matemàticament el coneixement clàssic del combat Pokémon** per servir com a base de dades i mètrica de comparació (*Baseline*).

Metodologia Heurística 1vs1 (Singles)

La lògica de decisió s'ha dividit en mòduls incrementals que l'agent avalua a cada torn:

- **Estadística i Dany:** Càlcul de dany estimat basat en la fórmula oficial, ponderant Atac/Defensa, bonificació per tipus (STAB) i resistències de l'oponent.
- **Consciència d'Estat (Utility):** Assignació de pesos extra a moviments que generen estats alterats (Burn, Paralysis) segons la vulnerabilitat del rival.
- **Lògica de Pivot (Switching):** Metodologia per decidir quan un canvi és millor que un "atac". Això es formalitza avaluant si el dany rebut en el proper torn serà superior a un llindar crític i si existeix un aliat al banc amb una resistència de tipus favorable.

Metodologia Heurística 2vs2 (Doubles)

El format de Dobles introdueix el problema de la **Coordinació Espacial**. No n'hi ha prou amb triar la millor acció per a cada slot individualment; cal que ambdues accions maximitzin la sinergia global. S'ha implementat el patró "**Score-then-Combine**":

1. **Fase de Scoring:** Es puntua cada possible moviment d'atac o canvi per als dos Pokémon per separat.
2. **Fase de Combinació:** L'agent avalua totes les combinacions possibles (fins a 81 en alguns torns) i aplica regles de sinergia:
 - **Focus Fire:** Si ambdós Pokémon ataquen el mateix objectiu, reben un bonus si la suma de dany garanteix un KO (evitant el malbaratament d'atacs).
 - **Protecció Selectiva:** Si un Pokémon està en una posició vulnerable (poca vida o desavantatge de tipus), el sistema avalua si "Protegir-se" mentre l'aliat ataca és més rendible que atacar directament.
 - **Mitigació de Residus:** Penalització si ambdós Pokémon ataquen a un rival que moriria amb un sol atac, incentivant que el segon Pokémon ataquí a l'altre oponent.

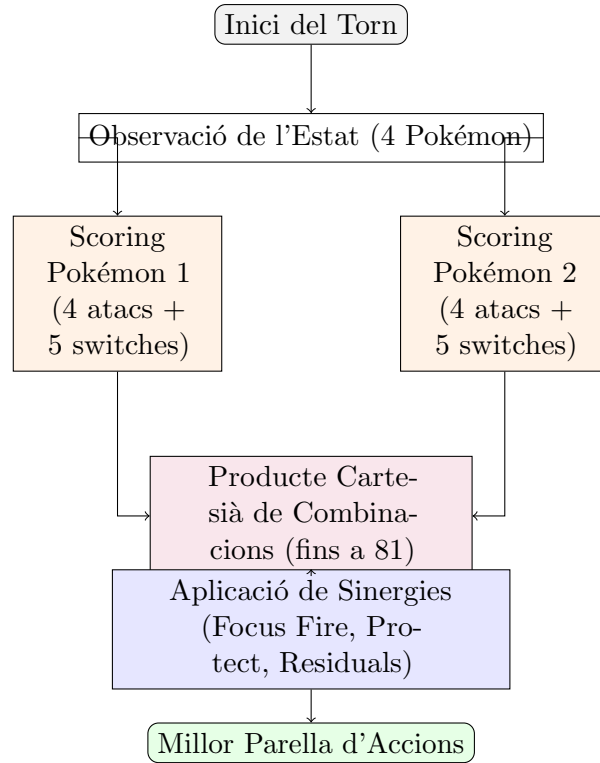


Figura 3.4: Flux de decisió del motor *Score-then-Combine* per a format Dobles.

3.6.2 Fase 2: Models d'Aprenentatge Profund i Raonament per LLM

Una vegada validada l'arquitectura i disposant del *Baseline Expert*, la segona gran fase de la recerca se centra en la substitució de la codificació dura per arquitectures d'aprenentatge o raonament.

Raonament per Models de Llenguatge (LLM)

La inclusió de LLMs com *Qwen 8B* respon a una hipòtesi de recerca: pot un model de llenguatge generalista aplicar heurístiques de combat sense haver estat entrenat exclusivament per a Pokémon? Metodològicament, això s'aborda mitjançant:

- **Inferència Local amb Ollama:** Per evitar el biaix de latència i el cost de les APIs de tercers, s'utilitza *Ollama* com a backend local. Això permet que el raonament de l'agent sigui privat, cost-zero i altament reproducible.
- **Chain of Thought (CoT):** No només es demana l'acció final, sinó que s'incentiva que el model "pensi" en veu alta (Strategy Logs). Això permet una avaluació qualitativa de la metodologia: l'agent ha triat el moviment correcte pel motiu correcte?

Reinforcement Learning (PPO)

L'eix científic del TFM assoleix el seu màxim pragmatisme avaluant si algorismes com PPO poden descobrir per si sols les normes causals que la Fase 1 programava explícitament (Taula de Tipus, STAB, predicció de canvis).

3.6.3 Línies de Control (Baselines)

Amb l'entorn escalable definit, cal fixar les línies base de referència sobre les quals mesurar tot progrés. S'utilitzen dos conjunts de baselines complementaris.

Baselines nadius de poke-env. Dins de poke-env [17] es proporcionen tres agents de referència que funcionen en **qualsevol generació i format** (Singles i Doubles):

1. **RandomPlayer**: Efectua moviments a l'atzar entre les accions legals. Actua com a referència mínima: qualsevol agent amb un mínim d'intel·ligència hauria de superar-lo àmpliament.
2. **MaxBasePowerPlayer**: Un agent cobdiciós (*Greedy*) que escull sempre l'atac amb la potència base més alta, ignorant tipus, STAB i context defensiu. Representa la barrera inferior d'estratègia funcional.
3. **SimpleHeuristicPlayer**: El rival més sofisticat ofert per la llibreria. Incorpora heurístiques preprogramades: avalua debilitats de tipus, prioritza canvis (*switches*) quan el seu Pokémon actiu es troba en desavantatge, i maximitza el dany considerant matchups. Superar aquest agent de manera consistent certifica que les noves funcions pròpies han assolit una intel·ligència superior.

Baselines externs de Pokechamp. Des del repositori Pokechamp [12] s'han integrat agents addicionals, principalment orientats al format **Singles de Generació 9** (`gen9randombattle`):

1. **AbyssalPlayer**: Agent heurístic avançat que avalua matchups de tipus, estima el dany esperat i decideix substitucions basant-se en l'anàlisi de tot l'equip rival. Incorpora lògica específica de Gen 9 (Teracristal·lització). Funciona com a rèplica millorada del SimpleHeuristicPlayer amb coneixement generacional.
2. **SafeOneStepPlayer**: Agent amb estimació de dany a un torn vista (*one-step lookahead*): puntua cada moviment per potència $\times$ STAB $\times$ efectivitat $\times$ precisió i escull el millor. S'ha reimplementat internament per evitar dependències del simulador local de Pokechamp que causaven bloquejos.
3. **Pokechamp i PokeLLMon (LLM)**: Agents basats en Models de Llenguatge (descrits al Capítol 2) que utilitzen raonament textual per prendre decisions. Requereixen un backend LLM local (detalls d'implementació al Capítol 5). Aplicables exclusivament a **Singles Gen 9**.

El contrast sistemàtic contra **ambdós conjunts** de baselines — nadius i externs — constitueix el filtre fonamental de la metodologia, a través del qual qualsevol avanç real serà rigorosament validat. Els capítols d'Implementació (Cap. 5) i de Resultats (Cap. 6) despleguen com s'han implementat aquests agents i quins avantatges o limitacions mostren respecte els models proposats en aquest TFM.

Capítol 4

Entorn de Desenvolupament i Infraestructura de Computació

El desenvolupament d'un projecte de recerca basat en *Deep Reinforcement Learning* i simulació massiva d'entorns de joc exigeix una infraestructura tecnològica robusta i evolutiva. Aquest capítol detalla la configuració del sistema de treball, des de les etapes inicials de desenvolupament local fins a la consolidació d'un entorn de computació remot d'alt rendiment, justificant les decisions tècniques preses en cada fase.

4.1 Evolució Cronològica de l'Arquitectura

L'entorn de treball ha passat per dues fases principals, marcades per la necessitat d'incrementar la potència de càlcul i la capacitat de gestió de memòria.

4.1.1 Fase 1: Desenvolupament Local i Prototipatge (Node Client)

Inicialment, el projecte es va prototipar en un ordinador portàtil d'alta gamma. Malgrat les seves prestacions, el format portàtil presentava limitacions en la gestió tèrmica i la quantitat de memòria volàtil (RAM).

- **Especificacions Tècniques:** Processador Intel[®] Core[™] Ultra 7 155H (22 fils d'execució) i 16 GB de memòria RAM.
- **Gestió de l'Emmagatzematge (Bind Mounts):** Amb un disc principal d'1 TB, es va decidir separar físicament les dades de l'execució. Es va utilitzar un segon disc NVMe d'1 TB dedicat exclusivament a la carpeta `data/`. Per evitar problemes de visibilitat amb Git que solen ocórrer amb els enllaços simbòlics, es va configurar un **bind mount** a nivell de sistema operatiu, permetent que el sistema de fitxers tractés el disc de dades com una carpeta local nativa dins del repositori.
- **Optimització de Codi i RAM:** Amb només 16 GB de RAM, l'execució de múltiples instàncies de *Pokémon Showdown* (Node.js) i l'orquestrador Python requeria un control rigorós. Moltes de les rutines de neteja manual del *Garbage Collector* (GC) de Python i els reinicis controlats de servidors detallats al Capítol 5 es van dissenyar originalment per sobreviure en aquest entorn limitat.

4.1.2 Fase 2: Consolidació en Servidor de Torre (Node Remot)

Un cop la infraestructura de simulació va ser estable, es va migrar l'execució principal a una estació de treball de sobretaula (*workstation*), configurada com a servidor de càlcul.

- **Especificacions Tècniques:** Processador AMD Ryzen™ 7 5700X3D (16 nuclis d'alt rendiment), 32 GB de memòria RAM DDR4 i una GPU NVIDIA RTX 2080 (8 GB VRAM).
- **Avantatges de Maquinari:** L'arquitectura de torre permet una dissipació tèrmica superior mitjançant refrigeració activa i dissipadors de grans dimensions, eliminant el risc de *thermal throttling* present al portàtil. A més, disposa de font d'alimentació dedicada i un Sistema d'Alimentació Ininterrompuda (SAI) per garantir la continuïtat de les simulacions davant talls elèctrics.
- **Integració de Recursos:** A diferència del portàtil, en aquest node tot el sistema (SO, execució i dades) coexisteix en una unitat NVMe d'alt rendiment, simplificant l'estructura de rutes i maximitzant l'amplada de banda d'I/O.

4.2 Gestió de Dependències i Entorn Python: uv

Per mantenir la consistència entre ambdós nodes i evitar la instal·lació innecessària de binaris pesants (com els *drivers* de CUDA) al portàtil, s'utilitza el gestor de paquets **uv** amb una configuració de dependències segmentada.

Al fitxer `pyproject.toml`, s'han definit grups de dependències opcionals (*extras*):

- **rl:** Llibreries base de Reinforcement Learning (Gymnasium, Stable-Baselines3).
- **gpu:** Versions específiques de PyTorch compilades amb suport per a CUDA, *torchaudio* i *torchvision*.
- **pokechamp:** Mòduls d'integració amb LLMs (Ollama, OpenAI, Transformers).

L'ús de **uv** permet utilitzar índexs diferenciats per a la instal·lació: mentre que el portàtil utilitza l'índex CPU de PyTorch per estalviar espai i recursos, el servidor utilitza l'índex `cu124`. Aquesta estratègia garanteix que les versions de les llibreries siguin idèntiques (v0.11.0 de *poke-env*, etc.), assegurant la reproductibilitat de l'experimentació independentment del maquinari.

4.3 Seguretat i Connectivitat Remota

La interacció amb el servidor es realitza de forma totalment remota des del portàtil mitjançant una combinació de tecnologies de xarxa mesh i IDEs moderns.

- **Tailscale SSH:** S'ha configurat una xarxa privada virtual (*overlay network*) basada en WireGuard que permet la connexió punt a punt entre els nodes. Tailscale SSH substitueix la gestió manual de claus per un model d'identitat centralitzat, eliminant la necessitat d'obrir ports al tallafocs (perillós en SSH convencional) i permetent superar el NAT sense configuració addicional.
- **Antigravity IDE i Remote-SSH:** S'utilitza l'extensió *Remote-SSH* per programar directament sobre el sistema de fitxers del servidor. Aquest flux de treball és superior al mètode de *git push/pull* constant, ja que permet provar canvis en temps real sense generar commits innecessaris i sense fragmentar la integritat del codi Font.
- **Persistència amb tmux:** Totes les execucions de llarga durada es llancen dins de sessions de **tmux**. Això garanteix que, si la connexió de xarxa entre el portàtil i el servidor s'interromp, els processos d'entrenament continuïn executant-se sense afectació.

4.4 Metodologia i Control de Versions

L'organització del projecte segueix principis de desenvolupament àgil i gestió estructurada:

- **Git i GitHub:** S'utilitza Git per al control de versions a nivell local i GitHub com a repositori centralitzat. Això no només facilita la sincronització de scripts entre el portàtil i el servidor, sinó que actua com a xarxa de seguretat per a l'historial del projecte.
- **Gestió d'Objectius (Kanban):** S'utilitza la plataforma **Trello** organitzada en un tauler Kanban (To Do, Doing, Done). Aquesta metodologia ha estat fonamental per segmentar el desenvolupament de les sis versions d'heurístiques, la implementació del pipeline de RL i la redacció d'aquesta memòria en unitats de treball gestionables.

4.5 Qualitat de Codi i Eines de Suport

Finalment, el projecte integra eines d'anàlisi estàtica per assegurar la robustesa del programari:

- **Ruff:** Utilitzat com a linter i formatejador unidireccional d'alta velocitat.
- **Ty:** Checker de tipus estàtic per a Python que detecta inconsistències lògiques abans de l'execució.
- **LaTeX Workshop:** Entorn de producció d'aquest document, integrat en el mateix IDE de desenvolupament per a una transició fluida entre la ciència de dades i la redacció acadèmica.

Capítol 5

Implementació d'Agents: De les Fases Heurístiques a l'Aprenentatge Profund

D'acord amb la metodologia evolutiva exposada al Capítol 3 i l'entorn infraestructural detallat al Capítol 4, aquest apartat detalla l'arquitectura de software construïda per donar vida als agents racionals dins del simulador *Pokémon Showdown*. Es cobreixen la infraestructura d'orquestració, l'evolució de les heurístiques, la integració d'agents externs i LLMs, i la implementació del pipeline de Reinforcement Learning. En primer lloc es presenta una visió global de les contribucions de software pròpies, i posteriorment s'entra en el detall de cada mòdul.

5.1 Contribucions de Software del Projecte

Abans de descriure el funcionament intern de cada component, es resumeixen les principals peces de software desenvolupades específicament en el marc d'aquest TFM:

- **Orquestrador paral·lel de servidors Showdown:** Scripts a `src/p03_scripts/` per llançar múltiples instàncies locals de Showdown, distribuir batalles de forma asíncrona i gestionar *checkpoints* i reinicis controlats.
- **Família d'heurístiques internes per Singles:** Agents a `src/p01DUMMY_UNDERSCOREheuristics/s01_si` (v1–v6) que implementen estratègies cada vegada més sofisticades basades en Taula de Tipus, STAB, status, boosts, prioritat i regles de *switching* defensiu.
- **Heurístiques preliminars per Dobles VGC:** Arquitectura *Score-then-Combine* per al format 2 vs 2, situada a `src/p01DUMMY_UNDERSCOREheuristics/s02DUMMY_UNDERSCOREdoubles/`, actualment en fase de finalització.
- **Reimplementació de SafeOneStepPlayer:** Versió pròpia de l'agent *OneStepPlayer* de Pokechamp, que elimina dependències de simuladors locals i opera exclusivament sobre l'API de *poke-env*.
- **Pipeline complet de Reinforcement Learning:** Mòduls a `src/p02_rl_models/` que inclouen el `StateDUMMY_UNDERSCOREVectorizer`, l'entorn `PokemonMaskedEnv` amb *action masking*, la funció de recompensa i la configuració de *MaskablePPO*.
- **Infraestructura de logging i benchmarking:** Escripcions sistemàtiques de resultats numèrics i, en el cas dels agents LLM, logs de raonament textual per a anàlisi posterior.

Aquests components es combinen amb llibreries i agents existents (poke-env, Pokechamp, sb3-contrib) que es fan servir com a fonaments sobre els quals construir i avaluar les contribucions pròpies.

5.2 Arquitectura Master-Worker per a Benchmarking Massiu

Com s'ha introduït a la Secció 3.1.5, l'execució de 10.000 partides per matchup requereix una infraestructura paral·lela robusta. Aquesta secció detalla l'arquitectura implementada, les decisions de disseny i els mecanismes de tolerància a fallades.

5.2.1 Visió General del Patró

El sistema segueix un patró **Master-Worker** clàssic adaptat a les particularitats del simulador Pokémon Showdown:

- **Master** (`benchmark.py`): Procés Python principal que itera sobre la matriu de matchups, gestiona la cua de ports disponibles, llança workers i consolida resultats.
- **Workers** (`worker.py`): Subprocessos Python efímers, cadascun amb el seu propi event loop `asyncio`, que executen un lot de batalles contra un servidor Showdown dedicat.
- **Servidors Showdown**: Instàncies Node.js independents, cadascuna escoltant en un port TCP exclusiu (8000, 8001, ..., 800N).

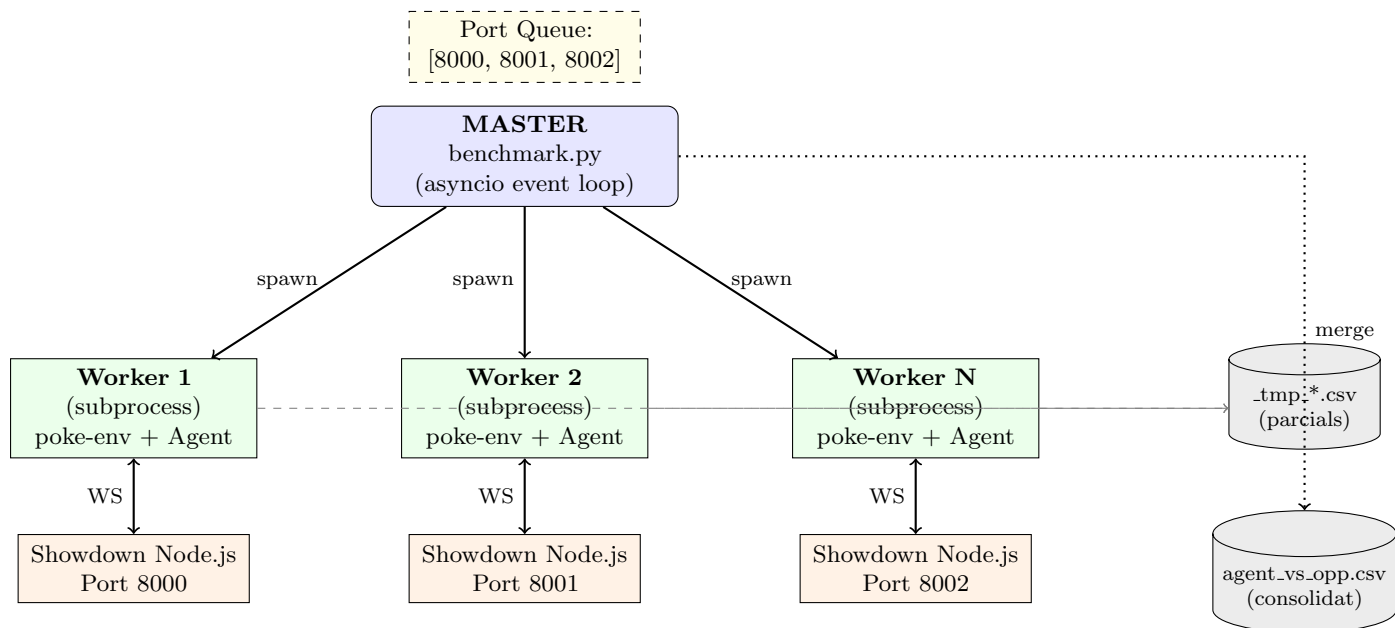


Figura 5.1: Arquitectura completa del sistema Master-Worker. Les línies contínues representen comunicació en RAM (processos i WebSocket), les discontinües representen escriptures a disc (persistència).

5.2.2 Paral·lelisme i Thread-Safety

La decisió d'utilitzar **multiprocessing** en lloc de **threading** és fonamental i respon a tres restriccions tècniques:

1. **Global Interpreter Lock (GIL):** El GIL de CPython impedeix l'execució paral·lela real de codi Python en múltiples fils. Threading només aporta beneficis per a operacions I/O-bound, però el càlcul de les heurístiques és CPU-bound.
2. **Aïllament d'estat `asyncio`:** Poke-env utilitza internament un event loop `asyncio` per gestionar les connexions WebSocket. Si s'utilitza `fork()` (el mode per defecte de `multiprocessing` en Linux), l'estat de l'event loop es copia al fill, causant *deadlocks*. Per evitar-ho, s'utilitza el context `spawn`:

```
_mp_ctx = multiprocessing.get_context("spawn")
```

Això crea un intèrpret Python completament nou per cada worker, sense herència d'estat.

3. **Alliberament garantit de memòria:** Quan un subprocés acaba (exit code 0 o error), el sistema operatiu reclama **tota** la RAM que ocupava. Això elimina qualsevol *memory leak* acumulat per poke-env, objectes `Battle`, o models LLM carregats en VRAM.

Cada worker opera sobre un port Showdown **exclusiu**. No hi ha cap recurs compartit entre workers: ni memòria, ni connexions de xarxa, ni fitxers (cada un escriu al seu propi `_tmp*.csv`). Això fa el sistema inherentment *thread-safe* sense necessitat de locks, semàfors o sincronització explícita.

5.2.3 Gestió de Memòria: RAM vs Disc

La Figura 5.2 mostra el cicle de vida de les dades durant una execució típica.

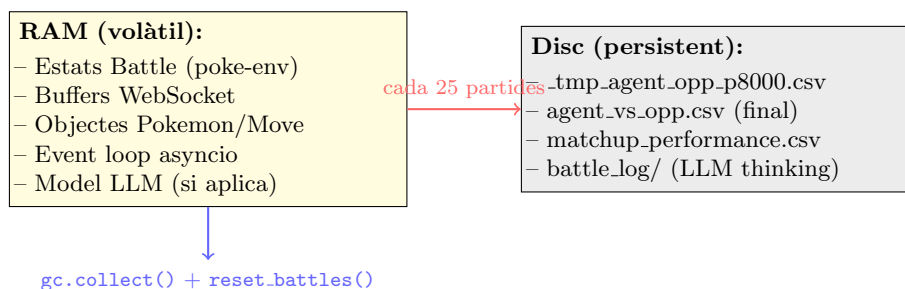


Figura 5.2: Cicle de vida de les dades: les batalles viuen en RAM fins que un chunk de 25 partides es completa, moment en què es serialitzen a disc i l'estat en memòria s'allibera.

Dins de cada worker, el cicle de gestió de memòria és:

1. Es juguen 25 batalles simultàniament (chunk) mitjançant `battle_against()`.
2. S'extreuen les mètriques de les batalles finalitzades i s'escriuen al CSV parcial.
3. Es crida `player.reset.battles()` i `opponent.reset.battles()` per eliminar les referències als objectes `Battle` acumulats.
4. Es crida `gc.collect()` per forçar la recollida d'objectes orfes.
5. Es repeteix fins a completar el lot assignat.

Sense aquest cicle, 10.000 batalles acumulades en un sol procés consumirien >2 GB de RAM. Amb el chunking, el consum es manté estable al voltant de ~200–400 MB per worker.

5.2.4 Tolerància a Fallades (Crash-Safe State)

L'execució de desenes de milers de partides pot trigar hores. El sistema implementa múltiples mecanismes per garantir que cap fallada causa la pèrdua del treball realitzat:

Recuperació Automàtica (Resume)

A l'inici de cada matchup, el master llegeix el CSV de sortida i compta les partides existents:

```
already_done = len(pd.read_csv(out_csv))
n_to_run = target_battles - already_done
if n_to_run <= 0: return # Matchup ja complet
```

Si l'execució es va interrompre (tall de corrent, error de xarxa, reinici del sistema), simplement re-executar el benchmark reprèn exactament on es va quedar. No cal cap flag `--resume` ni fitxers de checkpoint: la veritat rau en el CSV.

Aïllament de Fallades per Worker

Si un worker crasha (timeout, error de connexió, OOM):

- El seu fitxer temporal `_tmp_*.csv` es neteja sempre (bloc `finally`).
- El CSV principal **no es corromp**: les dades només s'hi afegeixen (*append*) un cop el worker ha completat satisfactòriament.
- El master detecta la falta de progrés i pot re-assignar les partides pendents en la següent iteració del bucle de reintents.

Timeouts Multinivell

El sistema implementa timeouts a tres nivells per evitar bloquejos indefinits:

Nivell	Timeout	Acció en cas d'expiració
Chunk (25 partides)	120–180s	S'extreuen els resultats parcials disponibles i es continua.
Worker complet	200s	El master mata el procés (SIGTERM) i allibera el port.
Detecció de progrés	–	Si un cicle complet no produeix cap partida nova, s'atura el matchup.

Taula 5.1: Timeouts multinivell del sistema de benchmarking.

5.2.5 Reinici Periòdic de Servidors

S'ha observat empíricament que els processos Node.js de Pokémon Showdown experimenten una degradació gradual de rendiment:

- **Causa:** El Garbage Collector de V8 (motor JavaScript) utilitza un algoritme generacional. Objectes de llarga durada (metadades de formats, estadístiques de Pokémon carregades en memòria) es promouen a l'*old generation*, on la recollida és menys agressiva. Això causa fragmentació de memòria que mai es recupera completament.
- **Síntoma:** Després de ~3.000–5.000 batalles, el procés Node.js creix de ~80 MB a >400 MB i la latència per torn augmenta un ~30%.
- **Solució:** Reinici periòdic controlat pel master.

El reinici s'activa cada N matchups **actius** (configurable amb `--restart-every`, per defecte 3). El procediment és:

1. `pkill -f pokemon-showdown` — Mata tots els processos Showdown.
2. Sleep 2 segons — Permet l'alliberament dels ports TCP.
3. Relançament de N instàncies noves via script bash.
4. Sleep 15 segons — Temps d'inicialització (càrrega de JSONs de Pokémon, moviments i habilitats a RAM).

El cost total és de ~ 17 segons de *downtime* per reinici. En una execució típica de 4 hores amb 30 matchups, això suposa menys de 3 minuts acumulats ($\sim 1\%$ del temps total), un cost negligible a canvi de la garantia d'estabilitat.

5.2.6 Paràmetres de Configuració

La Taula 5.2 resumeix els paràmetres configurables del sistema i els valors per defecte utilitzats en els experiments d'aquest TFM.

Paràmetre	Defecte	Descripció
<code>n_battles</code>	10.000	Partides objectiu per matchup.
<code>--ports</code>	2	Nombre de servidors Showdown paral·lels (workers simultanis).
<code>--start-port</code>	8000	Port base; s'assignen consecutivament (8000, 8001...).
<code>--concurrency</code>	10	Batalles simultànies per worker (<code>max_concurrent_battles</code>).
<code>--restart-every</code>	3	Reiniciar servidors cada N matchups actius.
<code>chunk_size</code>	25	Batalles per cicle d'escriptura a disc i GC.
<code>chunk_timeout</code>	180s	Timeout per chunk de batalles.
<code>batch_timeout</code>	200s	Timeout total per worker.

Taula 5.2: Paràmetres de configuració del benchmark i valors per defecte.

5.2.7 Dades Recollides per Partida

Cada batalla finalitzada genera un registre amb les mètriques següents, dissenyades per permetre una anàlisi multidimensional del rendiment dels agents:

Camp	Descripció
<code>battle_id</code>	Identificador únic de la batalla (assignat per Showdown).
<code>format</code>	Format jugat (p. ex. <code>gen9randombattle</code>).
<code>heuristic / opponent</code>	Noms dels dos agents enfrontats.
<code>won</code>	Resultat binari (1 = victòria de l'agent principal).
<code>turns</code>	Durada de la partida en torns.
<code>fainted_us / fainted_opp</code>	Pokémon derrotats per cada bàndol.
<code>remaining_pokemon_us/opp</code>	Pokémon supervivents al final.
<code>total_hp_us / total_hp_opp</code>	Suma de la fracció de HP dels supervivents.
<code>team_us / team_opp</code>	Composició completa dels equips (espècies, objectes, habilitats).
<code>move_stats_us / move_stats_opp</code>	Historial de moviments usats (<code>move_id:count</code>).
<code>crit_us / crit_opp</code>	Nombre de cops crítics rebuts/infligits.
<code>miss_us / miss_opp</code>	Nombre de moviments fallats.
<code>supereffective_us/opp</code>	Nombre de cops supereficients.
<code>voluntary_switches_us</code>	Canvis voluntaris (decisió estratègica).
<code>forced_switches_us</code>	Canvis forçats (per debilitació d'un Pokémon).

Taula 5.3: Camps recollits per cada partida durant el benchmarking. Permeten anàlisi de win-rate, eficiència, agressivitat, impacte de l’RNG i patrons de joc.

5.2.8 Instrumentació: StatsBattle

Per recollir les mètriques avançades (crits, misses, moviments per Pokémon), s’ha implementat una subclasse `StatsBattle` que intercepta els missatges WebSocket de Showdown abans que siguin processats per `poke-env`. Això s’aconsegueix mitjançant un *monkey-patch* dinàmic:

1. Al carregar-se, el worker substitueix el mètode `Player._create_battle()` per una versió que instancia `StatsBattle` en lloc de la classe base `Battle`.
2. `StatsBattle` sobreescriu `parse_message()` per detectar events com `|-crit|`, `|-miss|` i `|-supereffective|` i actualitzar comptadors interns.
3. Sobreescriu `switch()` per distingir entre canvis voluntaris (l’agent decideix canviar) i forçats (un Pokémon ha caigut i s’ha de triar substitut).
4. Al final de la batalla, els comptadors es serialitzen directament al CSV sense cap overhead addicional.

Aquesta tècnica és transparent per als agents: no necessiten cap modificació per funcionar amb `StatsBattle`, ja que la interfície de `Battle` es manté idèntica.

5.2.9 Workflow Complet d’una Execució

La Figura 5.3 mostra el flux complet d’una execució del benchmark, des de l’arrencada fins a la consolidació final.

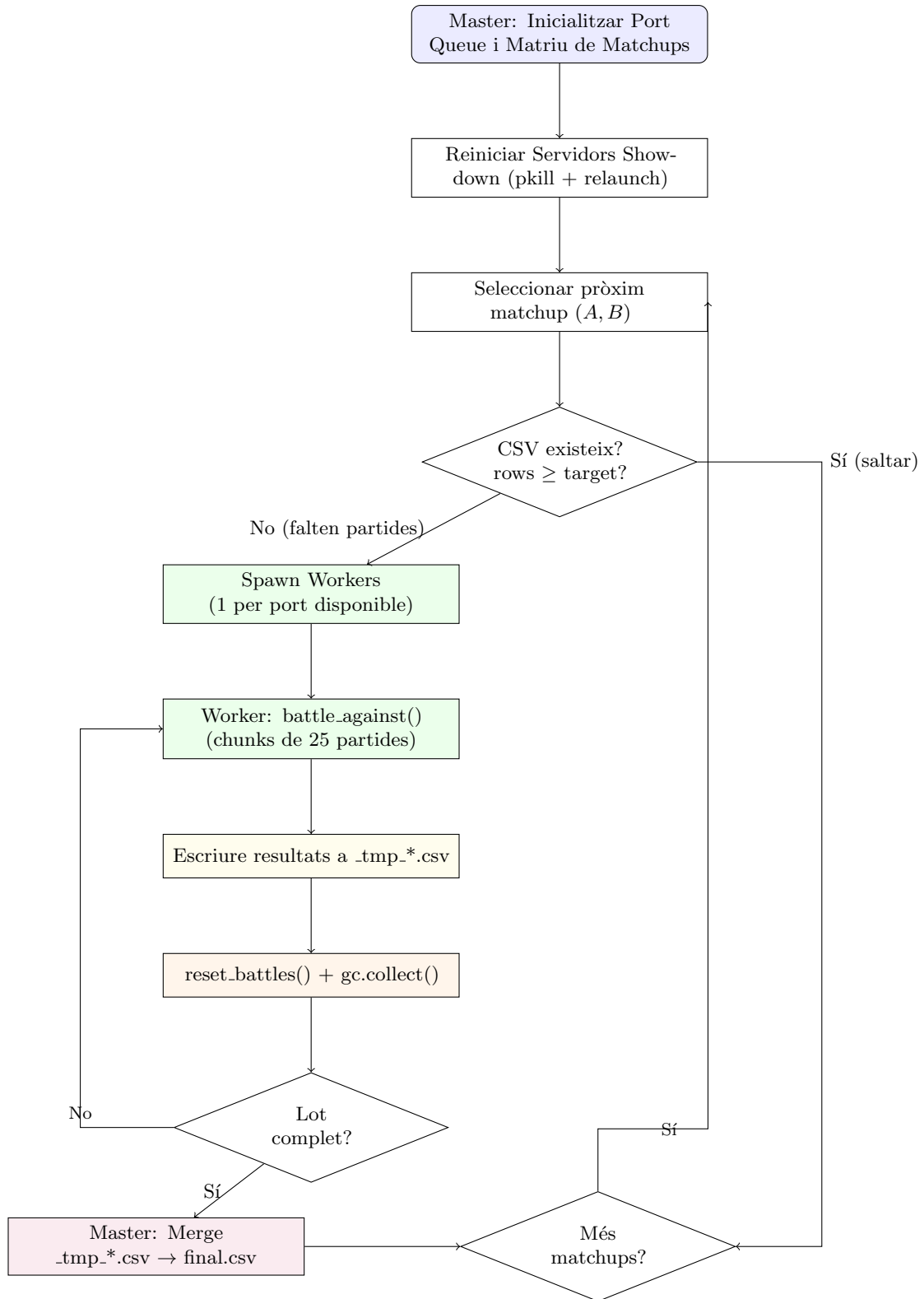


Figura 5.3: Flux complet d'execució del benchmark amb el cicle de recuperació, escriptura incremental i consolidació.

5.3 Fase 1.1: Desenvolupament d'Heurístiques per Singles (1vs1)

Abans de desplegar models d'aprenentatge, el projecte exigeix validar la infraestructura dins d'un espai d'accions reduït. El format **Singles** (1vs1) serveix com a arena de desenvolupament:

l'agent interactua contra un sol oponent visible, amb un màxim de 9 accions per torn (4 atacs + 5 substitucions). El resultat final d'aquesta fase és un agent heurístic intern “expert” que actua com a línia de control pròpia.

Mitjançant sis cicles evolutius de codi, s'han consolidat heurístiques progressivament més sofisticades sota `src/p01DUMMY_UNDERSCOREheuristics/s01_singles/agents/internal/`. A continuació es resumeix aquesta evolució, entenent-la com un procés iteratiu de millora gradual:

- **Heurística v1 (Naïve Aggressive):** Selecciona el moviment amb la potència base més alta, ignorant completament tipus, STAB i substitucions.
- **Heurística v2 (Damage Awareness):** Introdueix el càlcul de dany estimat, multiplicant la potència base pel factor d'efectivitat de tipus ($\times 0.5$, $\times 1$, $\times 2$, $\times 4$) i l'STAB ($\times 1.5$).
- **Heurística v3 (Defensive Pivot):** Afegeix la primera lògica defensiva. Si el Pokémon actiu està enverinat (Toxic) durant més de 2 torns, es substitueix. Si l'oponent és més ràpid i l'atac previst fa menys de 20 HP de dany, es busca un company millor. Aquesta lògica de *switching* és tan estable que les versions posteriors (v4–v6) la reutilitzen.
- **Heurística v4 (Field-Aware):** Refina el càlcul de dany incorporant el sistema de categories (físic vs. especial, considerant si l'atacant està cremat) i els modificadors climàtics i de terreny (Sol, Pluja, Electric/Grassy/Psychic Terrain).
- **Heurística v5 (Boost-Aware):** Estén v4 comptant les modificacions d'estadístiques en combat (*boosts*): considera els $+/-$ d'Atac, Defensa i Velocitat en la fórmula de dany, i afegeix regles de seguretat (substitucions preventives quan la vida pròpia és baixa).
- **Heurística v6 (Priority Master):** Integra totes les regles anteriors i afegeix un multiplicador de $\times 1.2$ als moviments de prioritat (*Quick Attack*, *Shadow Sneak*, etc.), permetent assegurar KOs contra oponents amb poca vida abans de rebre un atac.

La versió v6 constitueix la culminació d'aquesta família d'heurístiques i és la que es fa servir com a **baseline expert propi** en els experiments descrits al Capítol 3.

5.4 Fase 1.2: Integració d'Agents Externs (Pokechamp)

Per ampliar el conjunt de baselines i disposar d'agents externs contra els quals validar les heurístiques, s'ha integrat el repositori **Pokechamp** [12] al pipeline de benchmarking.

5.4.1 Agents Heurístics de Pokechamp (1vs1)

S'han incorporat els agents basats en regles de Pokechamp per al format Singles Generació 9:

- **AbyssalPlayer:** Agent heurístic que combina avaluació de matchups de tipus, estimació de dany amb STAB i efectivitat, i anàlisi de l'equip rival complet. Incorpora lògica específica de la Generació 9, incloent la Teracristal·lització.
- **SafeOneStepPlayer:** Reimplementació pròpia de l'agent *OneStepPlayer* original de Pokechamp. L'original utilitzava un simulador local (*LocalSim*) que causava bloquejos (*hangs*) per dependències de fitxers de cache absents. La nostra versió calcula $\text{dany} = \text{potència} \times \text{STAB} \times \text{efectivitat} \times \text{precisió}$ usant exclusivament l'API de `poke-env`, sense dependències externes.

5.4.2 Agents LLM: Pokechamp i PokeLLMon

Els agents basats en Models de Llenguatge (LLMs) representen un enfocament radicalment diferent: en lloc de regles codificades, utilitzen raonament en text natural per escollir accions.

Backend: Ollama amb Qwen 8B. Per executar els LLMs de forma local sense dependre d'APIs costoses, s'ha configurat **Ollama** [15] com a servidor d'inferència local. El model seleccionat, **Qwen 8B** [30], ocupa ~6–8 GB de VRAM i encaixa completament dins d'una GPU RTX 2080 (8 GB), permetent una inferència ràpida (~2–5 segons per torn). Models més grans (13B+) causaven *memory swapping* que alentia cada torn a minuts.

La configuració al benchmark s'especifica com:

```
--playerDUMMY\_UNDERSCOREbackend ollama/qwen3:8b
--playerDUMMY\_UNDERSCOREpromptDUMMY\_UNDERSCOREalgo io
--temperature 0.3
```

Arquitectura d'integració. L'agent Pokechamp rep l'estat de la batalla en format text, l'envia al LLM via API HTTP, i interpreta la resposta per generar una ordre de batalla. El flux és:

1. El benchmark llança un *worker subprocess* per cada batch de batalles.
2. L'AgentFactory detecta l'identificador (`pokechamp` o `pokellmon`) i importa dinàmicament `get_llm_player` del repositori Pokechamp clonat.
3. L'agent tradueix l'estat de batalla a un *prompt* textual i l'envia a Ollama.
4. El LLM genera un raonament (*Chain of Thought*) i una acció seleccionada.
5. L'acció es tradueix a una ordre de Showdown.

Logging de raonament. A diferència de les heurístiques, els agents LLM generen fitxers de “*Thinking*” que expliquen el raonament darrere de cada decisió. Aquests logs es guarden a `evaluation/results/LLM/` i permeten analitzar la qualitat del raonament estratègic del model (anàlisi de tipus matchups, predicció de moviments rivals, gestió de recursos, etc.).

Aïllament de processos. El principal repte tècnic va ser la gestió de memòria: el fil de fons `POKEDUMMY_UNDERSCORELOOP` de Pokechamp manté referències als objectes de la batalla fins i tot després de `del + gc.collect()`, acumulant memòria de forma il·limitada. La solució va ser l'**aïllament per subprocessos**: cada batch de batalles s'executa en un procés Python independent (`worker.py`), i quan el procés acaba, el sistema operatiu reclama *tota* la memòria.

5.5 Fase 1.2b: L'Ascens al Competitiu Real (Dobles VGC)

Tot repte d'IA queda incomplet si no aborda el format oficial de competició: **Dobles 2 vs 2**. La base d'heurístiques dissenyada per Singles resulta invàlida perquè la classe de `poke-env` per Dobles requereix retornar tuples d'accions coordinades per als dos Pokémon al camp. Dins de `src/p01DUMMY_UNDERSCOREheuristics/s02DUMMY_UNDERSCOREdoubles/` s'ha reestructurat l'arquitectura al patró *Score-then-Combine*:

5.5.1 Evolució i Refinament de les Heurístiques de Dobles

El desenvolupament de les heurístiques de Dobles ha estat un procés d'aprenentatge iteratiu. Inicialment, les versions es van anomenar de forma desordenada (v1, v2, v6, v7, v8). Posteriorment, vam realitzar una **reestructuració lògica** per reflectir la seva complexitat real:

- **v1 i v2:** Basades en la força bruta i dany estimat simple per Pokémon individual.

- **v3 (Environmental)** (Abans v6): Vam adonar-nos que el dany depenia críticament del clima i terrenys. Vam moure aquesta lògica a la V3 per establir una base sòlida de càlcul de dany.
- **v4 (Tactical Coordination)** (Abans v7): Vam introduir el *Focus Fire*. Vam descobrir que atacar a un sol rival sovint és més eficient que repartir el dany, encara que la potència total sigui menor.
- **v5 (Apex Strategy)** (Abans v8): La versió final que combina coordinació, gestió d'estats i proteccions selectives.

Aquest procés de "refactorització nominal" ens va permetre estandarditzar els benchmarks i millorar la llegibilitat dels resultats finals presentats al Capítol 6. A més, vam estandarditzar els fitxers de sortida: vam passar de noms complexos com `resultsDUMMY_UNDERSCORE2_vs_2_*.csv` a `win_rates_*.csv`, facilitant l'automatització dels scripts de visualització amb *Pandas*.

5.6 Pipeline de Processament de Dades i Analítica

Dada la gran quantitat de batalles executades (centenars de milers), s'ha dissenyat un pipeline de dades automatitzat que transforma els esdeveniments en brut del servidor en mètriques analítiques.

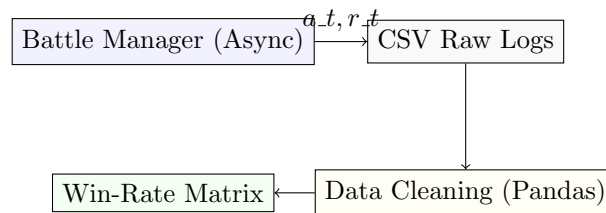


Figura 5.4: Pipeline de dades des de l'entorn de batalla fins a la generació de matrius.

El procés consta de tres etapes:

1. **Recol·lecció asíncrona:** Durant el benchmark, cada parella d'agents escriu una fila al fitxer `resultsDUMMY_UNDERSCORE*.csv` immediatament després d'acabar cada batalla. Això evita la pèrdua de dades si s'interromp el procés.
2. **Normalització nominal:** Els scripts de post-processament tradueixen els noms interns dels agents (ex. `HeuristicV4Doubles`) a noms amigables per al report (ex. `v4 (Focus)`).
3. **Agregació estadística:** Mitjançant *Pandas* [25], es calculen les mitjanes de victòries, desviacions i intervals de confiança, generant els fitxers `matrixDUMMY_UNDERSCORE*.csv` que alimenten les taules del TFM.

5.7 Fase 2: Implementació del Pipeline de Reinforcement Learning

Amb les heurístiques com a baselines experts, la segona gran fase del projecte ha consistit a construir un pipeline complet de Reinforcement Learning per entrenar agents neuronals, implementat a `src/p02_rl_models/`. Aquest pipeline materialitza, en codi, els conceptes de POMDP i PPO introduïts al Capítol 2: l'agent opera sobre observacions vectoritzades parcials de l'estat i actualitza una política estocàstica restringida per *clipping*.

5.7.1 Vectorització de l'Estat (StateDUMMY_UNDERSCOREVectorizer)

Les xarxes neuronals necessiten dades numèriques com a entrada. El mòdul `vectorizer.py` converteix l'objecte complex `Battle` de `poke-env` en un **vector float32 de 238 dimensions**, normalitzat a l'interval $[0, 1]$ per estabilitzar l'entrenament. Aquest vector actua com a observació parcial `o.t` del POMDP descrit al Capítol 2, capturant tota la informació rellevant que l'agent pot percebre a cada torn:

Component	Dims	Detall
Pokémon actiu propi	~33	HP normalitzat, tipus (18-dim multi-hot), status (one-hot), 7 boosts $[-6, 6] \rightarrow [0, 1]$.
Pokémon actiu rival	~33	Mateixa estructura que el propi.
Moviments propis	76	4 slots $\times$ (18 tipus one-hot + 1 potència normalitzada).
Banc propi	6	HP fraccional de cada Pokémon de l'equip.
Banc rival	7	HP dels Pokémon revelats + estimació de vius.
Clima i camp	~20	Weather i Field com a multi-hot encoding.
Condicions laterals	2×24	Hazards, pantalles, etc. per bàndol (normalitzats).

Taula 5.4: Estructura del vector d'observació de 238 dimensions generat pel `StateDUMMY_UNDERSCOREVectorizer`.

5.7.2 Entorn Gymnasium amb Action Masking

L'entorn `PokemonMaskedEnv` hereta de la classe `SinglesEnv` de `poke-env` i implementa la interfície estàndard de **Gymnasium**. L'espai d'accions és discret amb 10 accions possibles:

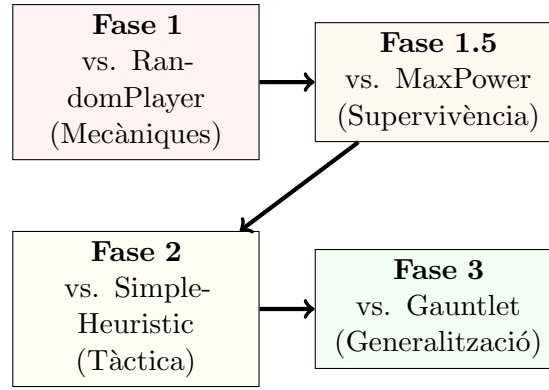
- **Accions 0–3:** Atacs (un per cada slot de moviment).
- **Accions 4–9:** Substitucions (un per cada posició del banc).

Per evitar que l'agent intenti accions il·legals (atacar amb un moviment sense PP, substituir per un Pokémon debilitat), s'utilitza **Action Masking**: cada torn, un vector binari indica quines de les 10 accions són legals. L'algorisme `MaskablePPO` [16] utilitza aquesta màscara per assignar probabilitat zero a les accions invàlides, reduint l'espai efectiu de cerca i estabilitzant l'aprenentatge en un entorn amb moltes accions contextualment prohibides.

5.7.3 Reward Shaping (Disseny de la Recompensa)

La funció de recompensa densa combina múltiples senyals per guiar l'aprenentatge:

- **Base:** Delta de HP (propis i rivals) + bonus per Pokémon derrotat (± 2) + victòria/derrota (± 30).
- **Boosts ofensius:** +0.3 per nivell de boost positiu en Atac, Atac Especial o Velocitat.
- **Penalització defensiva:** -0.1 per nivell de debuff propi.
- **Status oponent:** +0.3 si el rival actiu té un status debilitant (Burn, Paralysis, Toxic).
- **Hazards:** +0.5 per hazard plantat al camp rival; penalització proporcional per hazards propis.
- **Penalització temporal:** -0.02 per torn, per evitar partides infinitament llargues.



Checkpoint Transfer

Figura 5.5: Disseny del currículum d'entrenament per a l'agent RL.

5.7.4 Currículum d'Entrenament en 4 Fases

Per evitar que l'agent col·lapsi davant d'oponents massa complexos des de l'inici, s'ha implementat un **currículum progressiu** on la dificultat escala gradualment:

1. **Fase 1 — Fonaments:** Entrenament contra *RandomPlayer*. L'agent aprèn les mecàniques bàsiques: atacar és millor que no fer res, substituir un Pokémon debilitat és obligatori.
2. **Fase 1.5 — Supervivència:** Transició contra *MaxBasePowerPlayer*. L'agent ha d'aprendre a resistir un oponent que sempre ataca amb la potència màxima, desenvolupant nocions defensives.
3. **Fase 2 — Tàctica:** Transferència contra *SimpleHeuristicPlayer*. L'oponent ara avalua tipus i fa substitucions defensives, forçant l'agent a desenvolupar estratègia real.
4. **Fase 3 — Gauntlet:** Entrenament contra **tots els oponents simultàniament** (Random, MaxPower, SimpleHeuristic) amb reward shaping agressiu: victòria ± 100 i penalització temporal de -0.1 per torn, per evitar *stalling* i l'oblit catastròfic (*Cathedral Forgetting*).

En cada fase, es carrega el checkpoint de la fase anterior i es continua l'entrenament amb el nou oponent, permetent una **transferència progressiva** de coneixement.

5.7.5 Arquitectura del Model i Infraestructura GPU

L'agent utilitza **MaskablePPO** de la llibreria `sb3--contrib` [16] amb els següents hiperparàmetres:

- **Xarxa:** MLP amb 2 capes ocultes de 256 neurones cadascuna (`netDUMMY_UNDERSCOREarch=[256, 256]`).
- **Learning rate:** 3×10^{-4} .
- **Steps per batch:** 2048.
- **Entropy coefficient:** 0.01 (incentiva l'exploració en les primeres fases).
- **Discount factor (γ):** 0.99.

L'entrenament s'executa en una màquina amb una **GPU RTX 2080** (8 GB VRAM) i 16 nuclis de CPU. Per maximitzar el throughput, es despleguen **10 instàncies paral·leles** de Showdown (ports 8000–8009) amb `SubprocVecEnv`, permetent recopilar experiència 10 vegades més ràpid que amb un sol servidor.

5.8 Treball Futur: Aprenentatge Basat en Datasets de Partides

Més enllà del Reinforcement Learning pur implementat en aquest TFM, una línia de treball futur prometedora consisteix a explotar **datasets de partides existents** per entrenar agents mitjançant tècniques d'*Imitation Learning* o *Behavioral Cloning*. Aquesta secció descriu una proposta que **no s'ha implementat** en el marc d'aquest treball, però que es considera una extensió natural de l'arquitectura actual.

El repositori Pokechamp [12] inclou internament un conjunt de replays de partides competitives que registren seqüències completes d'estats i accions. La idea seria:

1. **Extracció:** Parsejar els replays per obtenir parelles $(s.t, a.t)$ d'estat-acció de jugadors experts.
2. **Supervisió:** Entrenar una xarxa neuronal per predir l'acció experta $a.t$ donat l'estat vectoritzat $o.t$ (usant el `StateDUMMY_UNDERSCOREVectorizer` ja implementat i la loss de cross-entropy).
3. **Fine-tuning amb RL:** Utilitzar el model pre-entrenat com a punt de partida per a les fases de RL, combinant el coneixement humà amb l'aprenentatge autònom.

Aquesta aproximació podria accelerar significativament la convergència del RL: en lloc de descobrir des de zero que “un atac de tipus Aigua és efectiu contra un Pokémon de Foc”, l'agent partiria d'un *prior* expert, i el RL només hauria de refinar les decisions en situacions ambigües o no representades al dataset.

5.8.1 Full de Ruta Immediat: Behavioral Cloning

Com a pas següent al RL pur, s'ha començat a preparar l'entorn per a l'**Imitation Learning**. L'objectiu és utilitzar els replays de jugadors humans d'alt nivell (disponibles a les bases de dades de Showdown) per pre-entrenar la política de l'agent.

La implementació requerirà:

1. Un script de *scraping* per descarregar milers de replays en format JSON.
2. L'adaptació del `StateDUMMY_UNDERSCOREVectorizer` per processar estats històrics a partir dels logs de text.
3. L'entrenament d'una xarxa de política usant una pèrdua de *Cross-Entropy* per minimitzar la diferència entre l'acció de l'humà i l'agent.

Capítol 6

Resultats i Avaluació de les Heurístiques

Aquest capítol presenta els resultats obtinguts de l'avaluació exhaustiva d'ambdós entorns de simulació (Singles i Doubles). S'analitza l'efectivitat i fiabilitat de les versions estructurades respecte als baselines, utilitzant les eines de *benchmark* internes de l'entorn Core, desenvolupades sota el disseny *Round-Robin* automatitzat explicat prèviament.

6.1 Infraestructura d'Execució: Gestió de Recursos

Un aspecte crític del procés de benchmarking massiu és la **gestió de memòria RAM**. Cada batalla Pokémon consumeix memòria tant al procés Python (objectes Battle, Players) com al servidor Showdown (processos `room-battle.js` de Node.js). Sense mecanismes de neteja, la memòria creix de forma il·limitada.

La Figura 6.1 mostra el perfil de consum de RAM durant l'execució d'un benchmark complet de Singles:

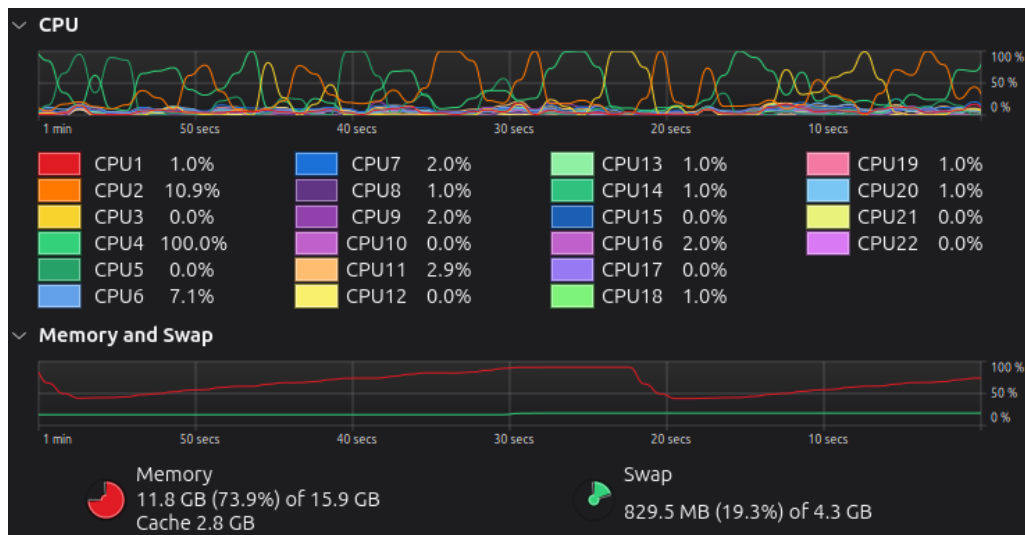


Figura 6.1: Consum de RAM durant un benchmark de Singles. S'observa com la memòria puja progressivament durant cada matchup (acumulació de batalles) i cau bruscament quan el servidor Showdown es reinicia i el Garbage Collector de Python s'executa entre grups de matchups.

El patró de dents de serra reflecteix el mecanisme de reinici controlat: cada N matchups **actius** (aquells on realment s'han executat partides i no s'han saltat per existència de dades

prèvies), l'orquestrador reinicia els servidors Showdown. Aquest refinament és crucial, ja que evita reinicis innecessaris quan es reprenen experiments gairebé completats, optimitzant el temps total d'execució en un factor de $\times 4$. Combinat amb l'aïllament per subprocessos, aquest sistema garanteix estabilitat total durant setmanes de computació ininterrompuda.

6.2 Avaluació Exhaustiva al Format 1 vs 1 (Singles)

L'anàlisi sistemàtic del format *Singles* s'ha ampliat per confrontar la línia completa d'heurístiques dissenyades (des de la versió *v1* fins a la *v12*) amb els agents basats en recerca de Pokechamp (*abyssal*, *one_step*, *safe_one_step*) i els baselines clàssics (*random*, *max_power*, *simple_heuristic*). L'avaluació comprèn un total de 18 agents lluitant en format Round-Robin complet, a raó de 10.000 partides per matchup en cadascuna de les 9 generacions de Pokémon Showdown (sumant més de 29,16 milions de partides simulades).

La Taula 6.1 mostra els resultats detallats de la matriu de percentatge de victòries (*Win Rate*) per a la Generació 9, la qual representa l'estat de l'art del format de Singles amb mecàniques complexes.

Taula 6.1: Matriu de Win-Rate (%) - Singles Gen 9

	(H) v12	(H) v11	(H) v10	(H) v9	(H) v8	(H) v7	(H) v6	(H) v5	(H) v4	(H) v3	(H) v2	(H) v1	(B) simple_heuristic	(C) abyssal	(C) safe_one_step	(C) one_step	(B) max_power	(B) random
(H) v12	50.1	61.7	67.4	61.5	67.3	68.5	76.1	74.9	75.8	75.5	76.3	76.3	59.8	59.9	76.5	76.4	90.3	98.6
(H) v11	37.8	50.2	54.2	50.5	54.3	55.5	64.6	64.4	65.2	65.7	64.8	65.2	47.3	47.5	64.3	65.7	87.6	98.5
(H) v10	33.2	45.9	49.3	46.7	50.5	50.7	62.0	61.5	61.1	62.7	62.6	63.0	42.7	41.9	62.2	62.0	87.2	97.7
(H) v9	37.7	49.4	53.8	50.3	53.6	55.1	64.7	64.5	63.7	64.3	64.7	65.1	47.0	46.6	65.5	65.5	87.9	98.4
(H) v8	32.9	46.0	50.8	46.8	49.9	51.6	62.2	61.4	61.0	61.6	62.1	61.8	42.8	42.3	62.1	61.4	87.0	97.7
(H) v7	32.0	46.1	48.4	45.3	48.8	50.2	61.3	59.0	59.7	60.5	61.5	61.1	40.7	40.8	60.5	60.9	85.7	97.6
(H) v6	23.9	35.1	37.9	36.1	38.8	39.3	49.8	49.1	49.5	50.5	50.6	51.5	32.0	32.0	50.1	50.8	80.5	97.4
(H) v5	24.8	35.1	39.5	36.4	38.7	40.1	52.1	50.1	50.7	51.1	51.1	51.8	31.1	32.5	51.0	51.2	80.5	97.6
(H) v4	24.4	35.6	38.7	35.1	39.4	39.7	51.3	49.7	50.1	51.4	50.7	52.0	31.5	31.3	51.3	50.3	80.9	97.3
(H) v3	24.4	34.2	37.6	35.4	38.2	38.6	48.8	48.5	48.9	50.8	50.9	49.9	31.5	31.1	49.8	50.6	80.0	97.1
(H) v2	24.1	34.7	37.4	35.0	36.7	37.5	49.6	48.7	49.1	49.6	50.1	50.3	31.4	31.2	50.3	50.5	79.9	97.3
(H) v1	23.3	34.4	36.5	35.1	37.4	39.0	48.8	47.8	47.6	49.1	49.1	50.4	30.3	30.6	48.8	49.9	80.2	97.5
(B) simple_heuristic	40.6	51.6	57.0	53.7	57.2	57.9	68.4	68.8	69.2	69.4	68.8	68.7	49.3	50.3	68.5	68.5	89.2	98.6
(C) abyssal	40.8	52.4	57.4	52.7	57.4	58.2	68.9	68.0	69.0	69.1	69.3	69.4	49.6	50.0	69.1	68.0	89.7	98.5
(C) safe_one_step	24.5	34.8	37.4	35.7	37.3	38.8	50.0	48.6	49.3	49.7	48.7	49.9	30.8	31.2	49.6	50.4	79.6	97.2
(C) one_step	23.5	34.9	37.0	34.9	37.9	39.2	48.7	49.4	49.4	50.5	50.2	50.2	31.1	31.5	49.3	50.3	80.0	97.4
(B) max_power	9.3	11.4	12.4	11.8	12.7	13.7	20.5	19.3	18.7	20.0	19.3	20.0	10.5	10.5	19.9	19.7	50.2	91.3
(B) random	1.8	1.7	1.9	1.8	2.3	2.1	2.8	2.3	2.6	2.6	2.8	2.4	1.4	1.5	2.4	2.5	9.1	50.1

Anàlisi de Dominància i la Matriu de Win-Rate

La matriu de victòries confirma la clara superioritat de l'heurística expert **v12 (The Meta Master)**. Aquest agent integra selecció de Pokémon inicial basada en matchups històrics (*lead selection*), canvi de Pokémon en debilitar-se basat en tipus (*fainted switch-in selection*), i ús estratègic de la *Terastal-lització*. Els resultats destaquen:

- La **v12** aconsegueix una taxa de victòria del **98,6%** contra l'agent *random* i un **90,3%** contra *max_power*.
- Supera de manera clara els millors baselines del projecte: un **59,9%** contra *abyssal* i un **59,8%** contra *simple_heuristic*.
- Manté taxes de victòria positives (superiors al 60%) contra totes les versions heurístiques pròpies anteriors (*v1* a *v11*), confirmant que cada millora incremental en la línia de disseny ha aportat un increment de rendiment net.

Anàlisi de l'Agressivitat: Diferencial de Fainted Pokémon

Per avaluar el grau de dominància de les victòries, la Taula 6.2 mostra el diferencial mitjà de Pokémon debilitats per partida (definit com a Pokémon rivals fainted – Pokémon propis fainted).

Un valor proper a +6 representa una victòria gairebé perfecta (*sweep*), mentre que valors propers a 0 reflecteixen enfrontaments molt anivellats. L'agent **v12** registra un diferencial de **+4,26** contra *random* i de **+2,62** contra *max_power*, indicant victòries extremadament solvents. Fins i tot contra els oponents més durs com *abyssal* i *simple_heuristic*, la **v12** manté un diferencial

Taula 6.2: Diferencial Mitjà de Pokémon Derrotats (Rival - Propi) - Singles Gen 9

	(H) v12	(H) v11	(H) v10	(H) v9	(H) v8	(H) v7	(H) v6	(H) v5	(H) v4	(H) v3	(H) v2	(H) v1	(B) simple_heuristic	(C) abyssal	(C) safe_one_step	(C) one_step	(B) max_power	(B) random
(H) v12		+0.03	+0.77	+1.04	+0.78	+1.04	+1.10	+1.58	+1.51	+1.54	+1.56	+1.57	+0.70	+0.68	+1.50	+1.59	+2.62	+4.26
(H) v11	-0.82		+0.01	+0.26	+0.01	+0.22	+0.28	+0.83	+0.81	+0.85	+0.88	+0.85	-0.13	-0.14	+0.85	+0.89	+2.39	+4.12
(H) v10	-1.02	-0.23		-0.02	-0.21	+0.00	+0.03	+0.60	+0.58	+0.57	+0.64	+0.65	-0.41	-0.46	+0.63	+0.60	+2.12	+3.83
(H) v9	-0.78	-0.03	+0.22		+0.02	+0.21	+0.30	+0.86	+0.82	+0.78	+0.81	+0.86	-0.16	-0.18	+0.86	+0.87	+2.39	+4.14
(H) v8	-1.04	-0.25	+0.03	-0.20		+0.00	+0.08	+0.64	+0.59	+0.58	+0.60	+0.62	-0.37	-0.41	+0.62	+0.59	+2.12	+3.84
(H) v7	-1.07	-0.24	-0.08	-0.26	-0.05		-0.00	+0.58	+0.50	+0.49	+0.57	+0.59	-0.50	-0.52	+0.54	+0.57	+2.04	+3.82
(H) v6	-1.58	-0.84	-0.61	-0.82	-0.61	-0.57		-0.01	-0.06	-0.01	+0.01	+0.02	-1.03	-1.05	+0.00	+0.03	+1.61	+3.53
(H) v5	-1.51	-0.84	-0.54	-0.80	-0.58	-0.51	+0.07		+0.00	+0.04	+0.06	+0.06	-1.04	-1.00	+0.04	+0.04	+1.64	+3.54
(H) v4	-1.54	-0.85	-0.56	-0.86	-0.57	-0.53	+0.06	-0.03		-0.00	+0.07	+0.04	-1.02	-1.06	+0.06	+0.03	+1.61	+3.54
(H) v3	-1.54	-0.89	-0.64	-0.82	-0.61	-0.59	-0.05	-0.09	-0.03		+0.03	+0.05	-1.03	-1.09	-0.01	+0.01	+1.58	+3.52
(H) v2	-1.57	-0.84	-0.64	-0.86	-0.69	-0.62	-0.03	-0.05	-0.07	-0.02		+0.02	-1.06	-1.07	+0.01	+0.03	+1.58	+3.51
(H) v1	-1.59	-0.90	-0.67	-0.86	-0.66	-0.58	-0.06	-0.10	-0.12	-0.02	-0.04		-1.10	-1.12	-0.06	-0.01	+1.60	+3.51
(B) simple_heuristic	-0.69	+0.06	+0.39	+0.17	+0.39	+0.44	+1.05	+1.06	+1.08	+1.09	+1.08	+1.07	-0.04	-0.00	+1.05	+1.04	+2.53	+4.14
(C) abyssal	-0.66	+0.15	+0.43	+0.15	+0.40	+0.47	+1.09	+1.02	+1.08	+1.08	+1.09	+1.09	-0.01	+0.01	+1.08	+1.05	+2.56	+4.14
(C) safe_one_step	-1.55	-0.85	-0.63	-0.80	-0.66	-0.58	-0.01	-0.09	-0.03	-0.00	-0.04	-0.00	-1.06	-1.09	-0.02	+0.02	+1.56	+3.51
(C) one_step	-1.58	-0.84	-0.64	-0.86	-0.63	-0.58	-0.05	-0.04	-0.04	+0.01	+0.01	+0.00	-1.07	-1.07	-0.04	+0.02	+1.59	+3.54
(B) max_power	-2.67	-2.42	-2.13	-2.44	-2.14	-2.08	-1.57	-1.64	-1.65	-1.60	-1.60	-1.58	-2.54	-2.55	-1.60	+0.00	+2.82	
(B) random	-4.21	-4.12	-3.85	-4.11	-3.84	-3.82	-3.51	-3.56	-3.54	-3.52	-3.52	-3.49	-4.15	-4.13	-3.56	-3.54	-2.79	+0.01

positiu de **+0,68** i **+0,70** respectivament, superant en mitjana a l'oponent per més de la meitat d'un Pokémon.

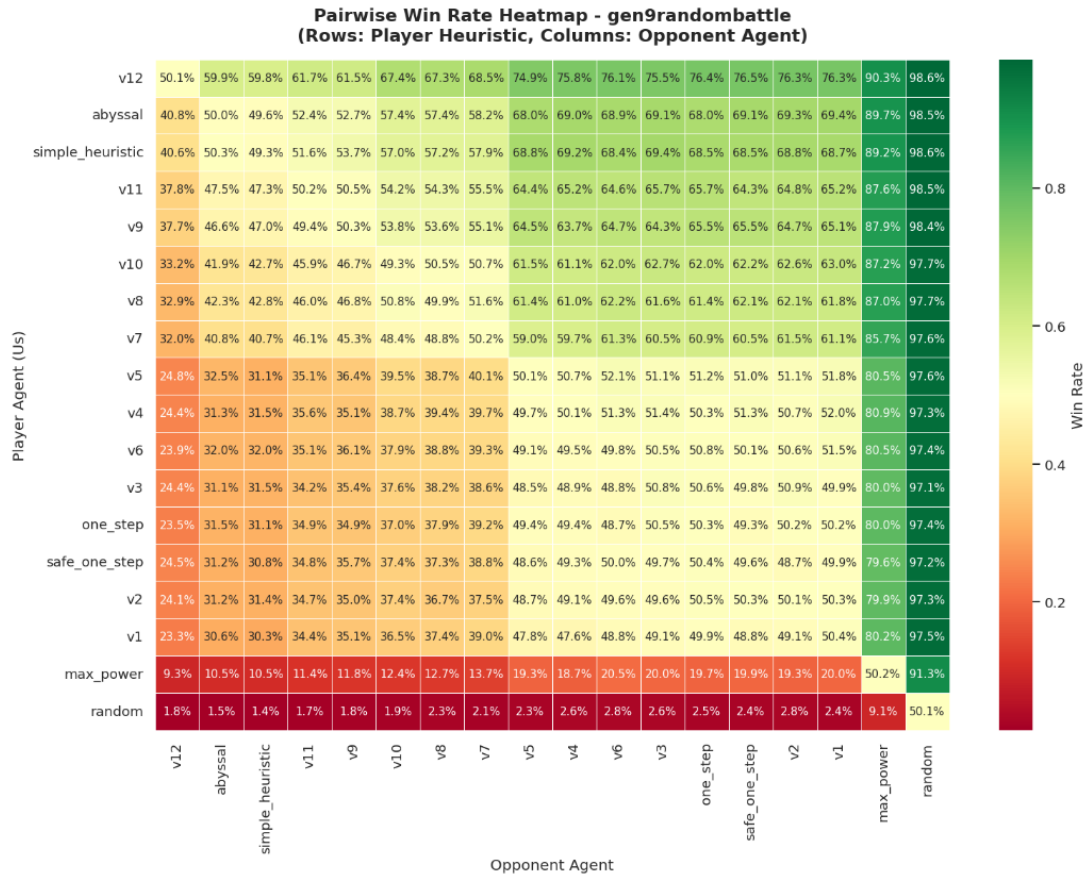


Figura 6.2: Matriu de calor mostrant el percentatge de victòries (*Win Rate*) de tots els encreuaments de *Singles* (Gen 9) avaluant baselines, heurístiques pròpies (v1-v12) i els agents experts de Pokechamp. S'observa la forta dominància del bloc de la versió v12.

Rànquing Elo Cross-Generacional (Generacions 1–9)

Per obtenir una mètrica unificada que reflecteixi la força competitiva de cada agent tenint en compte la qualitat dels oponents enfrontats, s'ha aplicat el model de Bradley-Terry via estimació per màxima versemblança (MLE) sense regularització. La Taula 6.3 presenta els ratings Elo calculats de forma independent per a cadascuna de les 9 generacions, ancorant el rendiment de l'agent *random* a exactament 1000 punts de referència.

Taula 6.3: Classificació i Rànquing Elo Bradley-Terry de tots els agents Singles (Generacions 1–9)

Agent	Gen 1	Gen 2	Gen 3	Gen 4	Gen 5	Gen 6	Gen 7	Gen 8	Gen 9
(H) v12	1846	1742	1751	1745	1771	1771	1771	1772	1817
(H) v11	1822	1716	1728	1716	1739	1738	1736	1736	1728
(H) v10	1823	1709	1709	1694	1716	1712	1707	1713	1702
(H) v9	1809	1713	1720	1709	1734	1733	1731	1731	1725
(H) v8	1821	1706	1700	1689	1711	1707	1705	1711	1701
(H) v7	1824	1710	1691	1677	1701	1696	1695	1703	1692
(H) v6	1775	1677	1660	1636	1653	1643	1636	1635	1619
(H) v5	1782	1676	1662	1638	1656	1648	1644	1642	1624
(H) v4	1781	1678	1662	1636	1655	1646	1641	1642	1622
(H) v3	1776	1677	1659	1632	1649	1640	1634	1632	1616
(H) v2	1776	1676	1659	1632	1649	1640	1634	1632	1616
(H) v1	1762	1673	1657	1630	1647	1638	1634	1634	1613
(B) simple_heuristic	1841	1763	1749	1744	1771	1768	1767	1892	1752
(C) abyssal	1841	1764	1750	1732	1758	1762	1762	1894	1752
(C) safe_one_step	1769	1695	1671	1634	1654	1641	1638	1637	1616
(C) one_step	1770	1694	1670	1634	1652	1642	1638	1636	1616
(B) max_power	1579	1510	1448	1430	1440	1421	1415	1383	1378
(B) random	1000	1000	1000	1000	1000	1000	1000	1000	1000

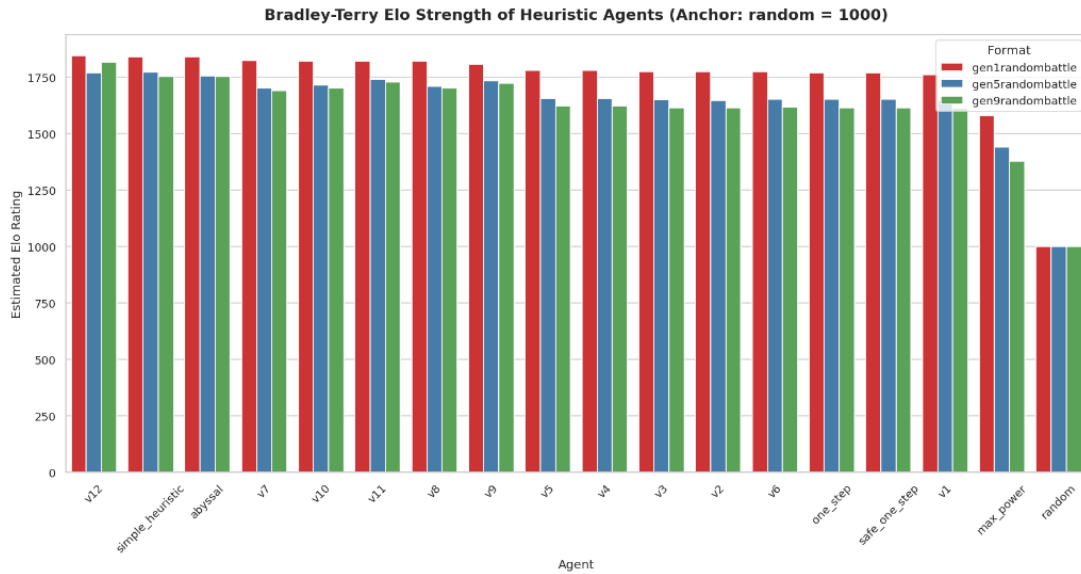


Figura 6.3: Rànquing Elo Bradley-Terry dels agents de Singles a les generacions representatives 1, 5 i 9. S'evidencia la bretxa competitiva de la v12 i la seva robustesa en entorns tant clàssics com moderns.

L'anàlisi de l'Elo revela conclusions clau:

- **La v12 és el líder indiscutible** en la generació moderna (Gen 9, 1817 Elo) i en la generació fundacional (Gen 1, 1846 Elo). A la Gen 1, supera lleugerament a *simple_heuristic* (1841 Elo) i a *abyssal* (1841 Elo).
- **Consistència a la Gen 5:** A la Generació 5 (l'era de la transició climàtica complexa), la **v12** s'enfronta a un metagame molt particular on el control del clima és crític. Aquí la **v12** obté 1771,2 Elo, empatant tècnicament amb *simple_heuristic* (1771,4 Elo) i superant a *abyssal* (1757 Elo).

- **La bretxa dels Baselines:** S'identifiquen clarament quatre nivells competitius:
 - *Tier 3 (Baselines bàsics):* *random* (1000 Elo) i *max_power* (mitjana de 1430 Elo).
 - *Tier 2 (Heurístiques bàsiques v1-v6):* Agrupats estretament al voltant d'uns 1615 Elo en Gen 9 i uns 1775 Elo en Gen 1.
 - *Tier 1 (Heurístiques avançades v7-v11):* On la introducció del maneig de trampes d'entrada (*hazards*), moviments de *setup* en torns lliures i lògica de sacrifici (*sack logic*) eleva el rendiment fins a la franja de 1690–1730 Elo en Gen 9.
 - *Tier 0 (Líders de Meta):* Dominat per la **v12**, *simple_heuristic* i *abyssal*.

6.2.1 Anàlisi de Metagames i Dinàmiques de Joc

L'avaluació massiva a través de les 9 generacions permet extreure dinàmiques de joc històriques de Pokémon. La Figura 6.4 analitza la durada mitjana de les batalles (en torns) i la taxa de canvis voluntaris realitzats pels jugadors per partida.

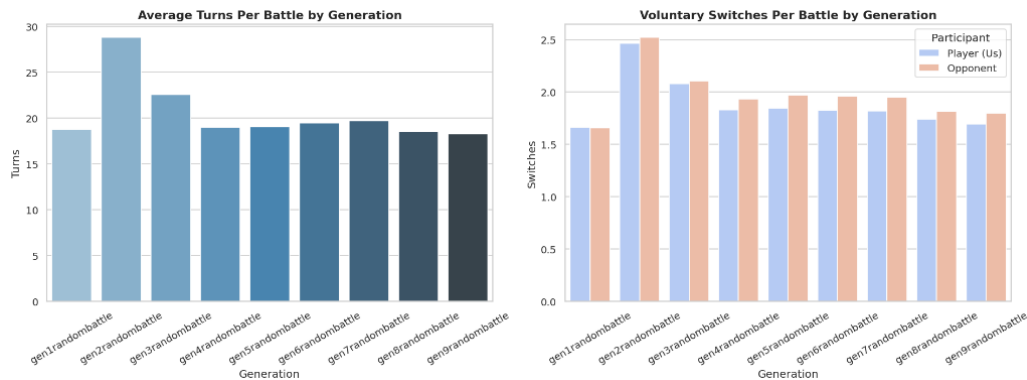


Figura 6.4: Durada mitjana de la batalla (torns) i taxa de canvis voluntaris per partida segons la generació de Pokémon Showdown. S'observa clarament l'efecte de stall de la Generació 2.

Les dades revelen dos comportaments macro-estratègics:

- **El fenomen de Stall de la Gen 2:** La Generació 2 (*gen2randombattle*) és amb diferència la més llarga de la història, amb una mitjana de **28,78 torns** per batalla, a més de registrar la taxa de canvis voluntaris més elevada de tot el dataset (**2,5 canvis per batalla**). Això es deu al fet que pràcticament tots els Pokémon en aquest format estan equipats amb l'objecte *Restes* (*Leftovers*), fet que atorga recuperació passiva constant, i alhora no existeixen objectes hiper-ofensius com la *Cinta Triada* o la *Vidaesfera* per trencar defenses fàcilment, forçant un metagame de canvis defensius constants.
- **Aceleració moderna (Gen 4–9):** A partir de la Generació 4, la durada mitjana s'estabilitza en un rang estret de **18 a 19 torns**, sent la Generació 9 la més ràpida de totes amb només **18,27 torns** de mitjana. L'augment de moviments ofensius d'alta potència i mecàniques explosives redueix dràsticament la viabilitat de les estratègies defensives pures.

6.2.2 Freqüència de Moviments i Popularitat de Pokémon

Mitjançant la projecció i l'anàlisi de freqüències dels logs de combat extrets de les 29,16 milions de partides, s'ha identificat quins són els moviments i els Pokémon més freqüents en el format de combats aleatoris. La Figura 6.5 reflecteix aquestes distribucions.

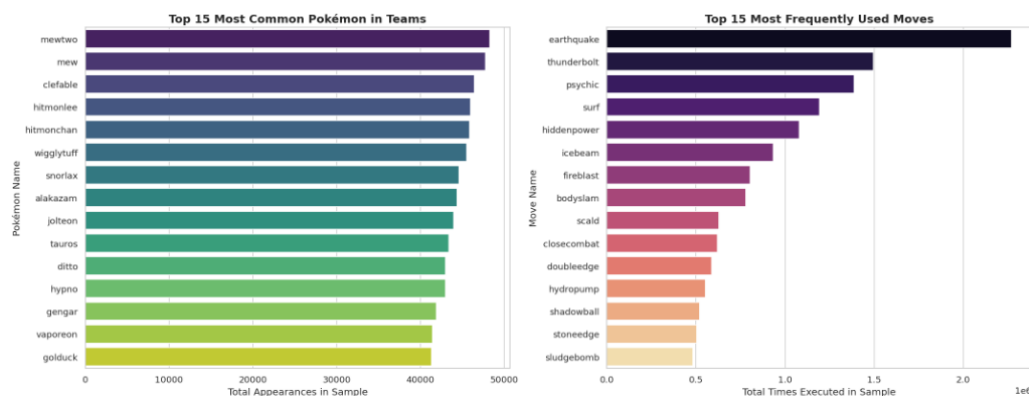


Figura 6.5: Distribució de freqüències dels 10 moviments més utilitzats (esquerra) i dels 10 Pokémon més convocats als equips (dreta) al llarg de tot el benchmark de Singles.

Els resultats confirmen:

- **Terratrèmol** (*Earthquake*) és el moviment més executat de tot el dataset, registrant més de **2,25 milions** d'usos. La seva alta potència base (100), precisió perfecta (100%) i excel·lent cobertura contra tipus molt comuns com Acer, Foc, Roca i Elèctric el converteixen en un moviment indispensable en qualsevol Pokémon físic.
- Pokémon singulars i llegendaris com **Mew** i **Mewtwo** apareixen en més de **47.000 equips** de la mostra, sent els personatges més recurrents en les alineacions aleatòries generades per Showdown.

6.3 Avaluació al Format 2 vs 2 (Dobles)

L'avaluació en el format de Dobles constitueix un dels reptes més importants d'aquest TFM. A diferència de Singles, on les heurístiques v1-v6 ja estaven consolidades, a Dobles s'ha realitzat un procés de recerca des de zero que ha culminat en la versió **v5 (Apex Heuristic)**.

S'han realitzat experiments en dues generacions clau per observar la consistència de les heurístiques:

Resultats a la Generació 9 (Format Actual). A la Figura 6.6 s'observa el rendiment de les 5 versions de Dobles en Gen 9. Es confirma que totes les versions superen el 80% de victòries contra **maxDUMMY_UNDERSCOREpower** i gairebé el 100% contra **random**. Les dades exactes es recullen a la Taula 6.4.

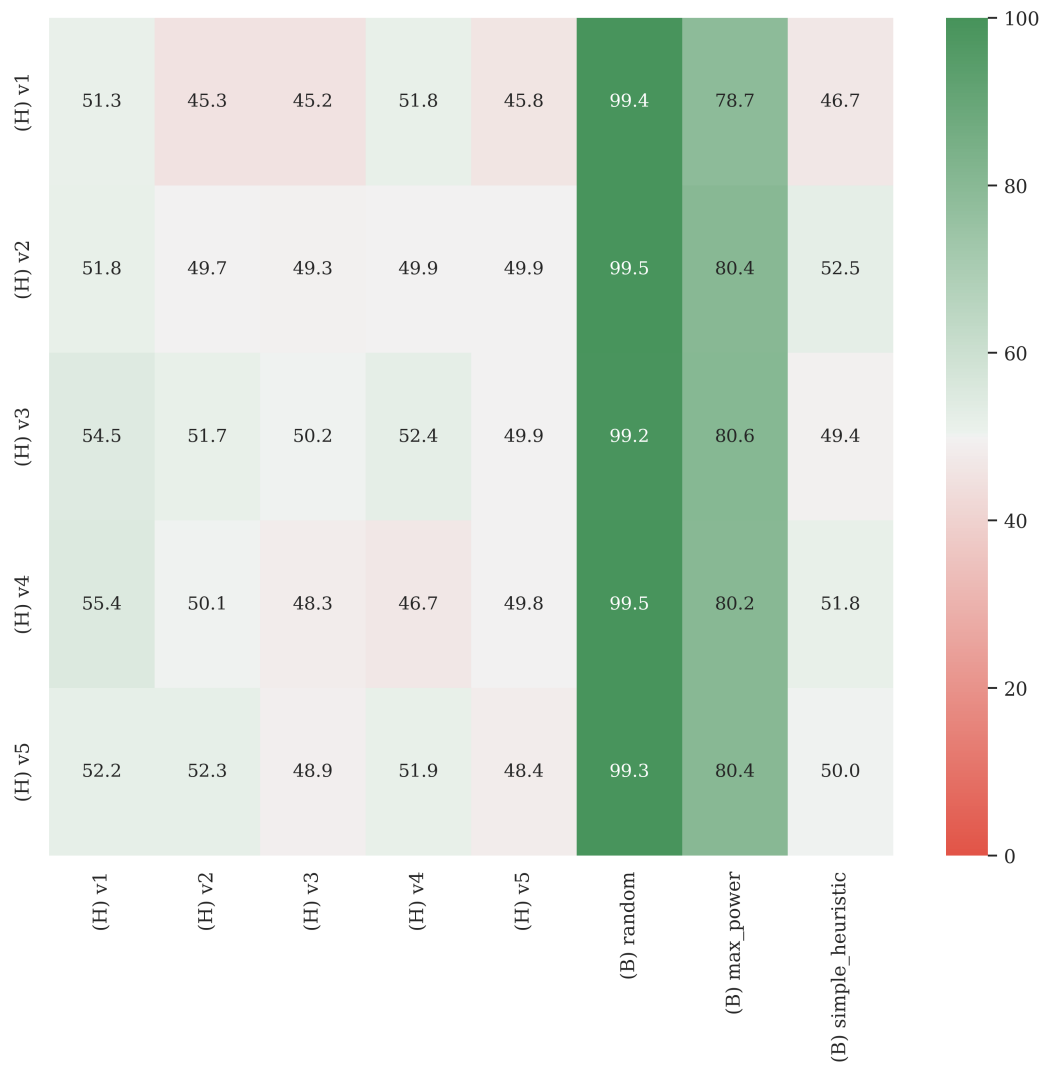


Figura 6.6: Matriu de win-rate per a *Random Doubles Battle* Generació 9.

Taula 6.4: Doubles Matchup Win-Rate Matrix (%)

	(H) v1	(H) v2	(H) v3	(H) v4	(H) v5	(B) random	(B) max_power	(B) simple_heuristic
(H) v1	51.3	45.3	45.2	51.8	45.8	99.4	78.7	46.7
(H) v2	51.8	49.7	49.3	49.9	49.9	99.5	80.4	52.5
(H) v3	54.5	51.7	50.2	52.4	49.9	99.2	80.6	49.4
(H) v4	55.4	50.1	48.3	46.7	49.8	99.5	80.2	51.8
(H) v5	52.2	52.3	48.9	51.9	48.4	99.3	80.4	50.0

Resultats a la Generació 8 (L’Era de la Coordinació). Els resultats a Gen 8 (Figura 6.7) són encara més reveladors. Aquí, les versions amb coordinació avançada (**v4** i **v5**) treuen un avantatge massiu a les versions més simples. La versió **v4 (Tactical Coordination)** es posiciona com la millor de tot el projecte en aquest format.

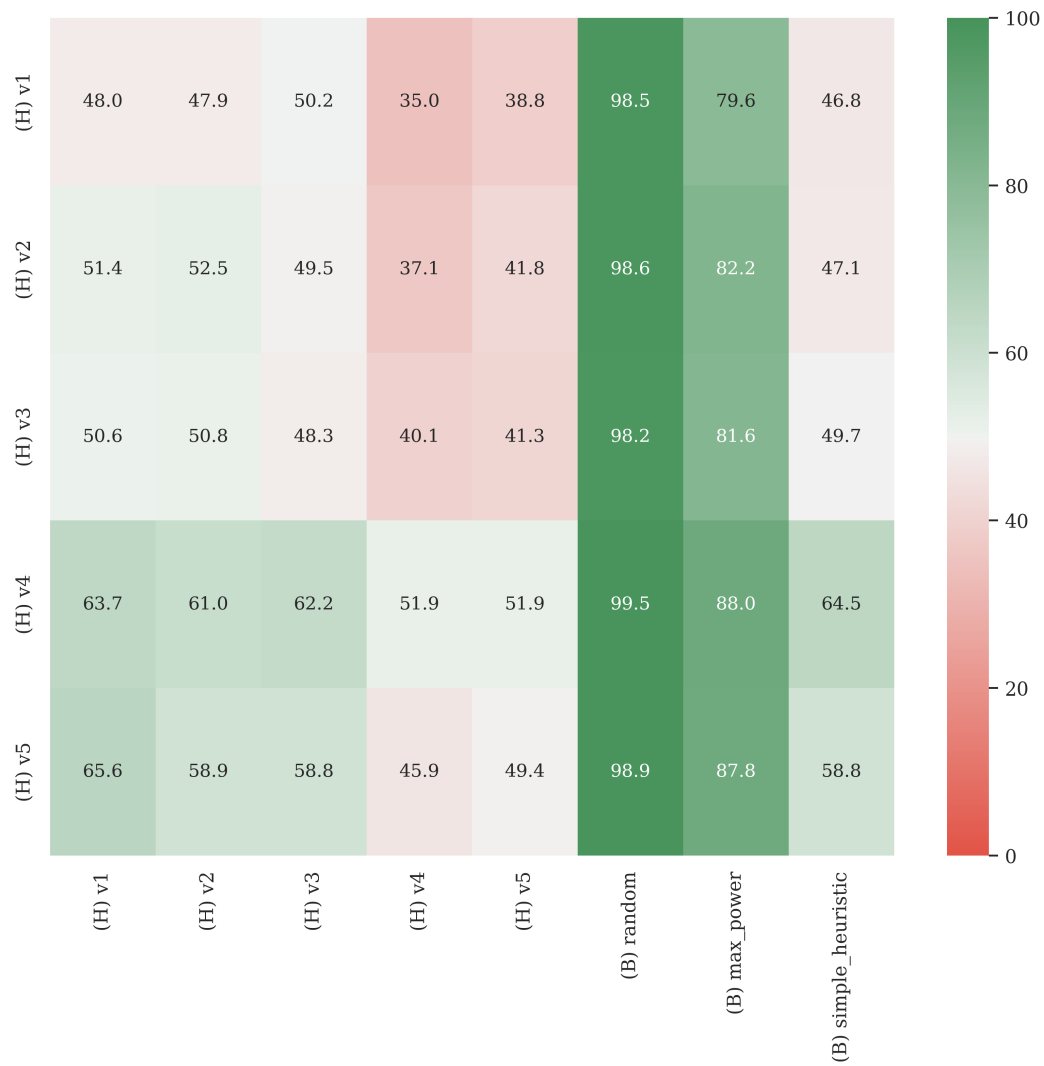


Figura 6.7: Matriu de win-rate per a *Random Doubles Battle* Generació 8. S’observa clarament com v4 i v5 dominen el quadrant superior dret.

A la Taula 6.5 es detallen els percentatges exactes de victòria per a la Generació 8, on el rendiment de la versió **v4** marca el rècord absolut de superioritat estratègica del projecte.

Taula 6.5: Doubles Matchup Win-Rate Matrix (%)

	(H) v1	(H) v2	(H) v3	(H) v4	(H) v5	(B) random	(B) max_power	(B) simple_heuristic
(H) v1	48.0	47.9	50.2	35.0	38.8	98.5	79.6	46.8
(H) v2	51.4	52.5	49.5	37.1	41.8	98.6	82.2	47.1
(H) v3	50.6	50.8	48.3	40.1	41.3	98.2	81.6	49.7
(H) v4	63.7	61.0	62.2	51.9	51.9	99.5	88.0	64.5
(H) v5	65.6	58.9	58.8	45.9	49.4	98.9	87.8	58.8

Comparativa Elo i Consistència Cross-Generacional

L’anàlisi de l’Elo en Dobles (Taules 6.7 i 6.6) revela la solidesa de l’arquitectura *Score-then-Combine*.

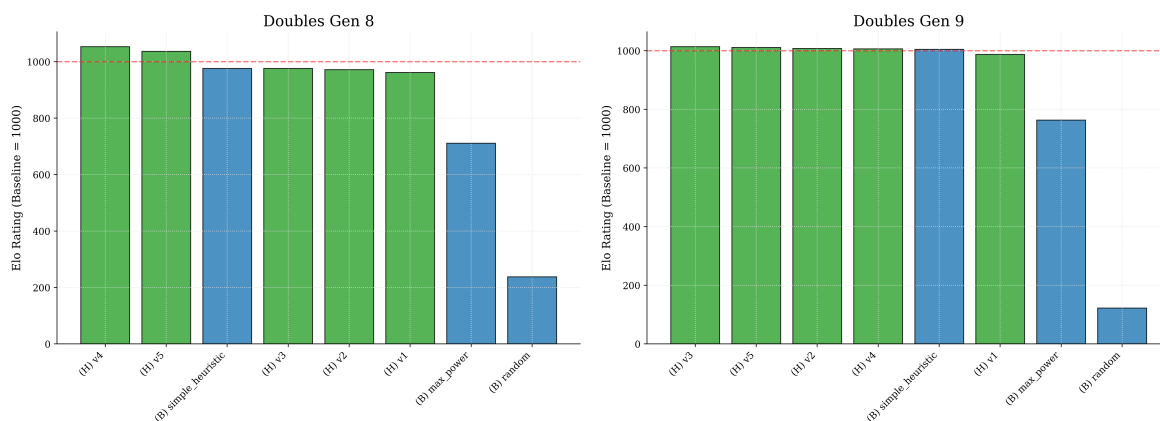


Figura 6.8: Comparativa de Rànquing Elo en Dobles: Generació 8 (esquerra) vs Generació 9 (dreta).

Taula 6.6: Elo Ranking - Doubles Gen 8

Agent	Elo Rating
(H) v4	1053
(H) v5	1036
(B) simple_heuristic	976
(H) v3	976
(H) v2	972
(H) v1	962
(B) max_power	711
(B) random	238

Taula 6.7: Elo Ranking - Doubles Gen 9

Agent	Elo Rating
(H) v3	1014
(H) v5	1011
(H) v2	1007
(H) v4	1006
(B) simple_heuristic	1005
(H) v1	987
(B) max_power	764
(B) random	122

S'observa que la versió **v4** és la més forta a Gen 8, mentre que a Gen 9 les versions **v3**, **v4** i **v5** es mantenen molt properes. Això suggereix que l'heurística de coordinació és fonamental en ambdós metagames, però l'impacte de mecàniques com el Dynamax a la Gen 8 feia que la coordinació tàctica de la v4 fos encara més discriminant que en el format actual.

Conclusió Científica: El canvi metodològic del 1vs1 al 2vs2 confirma que la coordinació (*Focus Fire* i *Protect*) és la clau per dominar el format de Dobles. Mentre que en Singles el dany pur és molt efectiu, en Dobles la capacitat de triar un objectiu comú per assegurar un KO és el que marca la diferència entre un agent mediocre i un expert.

Capítol 7

Conclusions

Aquest capítol sintetitza les fites assolides durant el desenvolupament del TFM.

7.1 Resum de Contribucions

S’ha construït una infraestructura robusta i paral·lela capaç d’executar milers de batalles simulades de Pokémon en formats tant de Singles com de Doubles. L’evolució de les heurístiques expertes ha demostrat que, encara que el format de Singles es pot dominar amb dany i pivots defensius simples, el format de Doubles exigeix una coordinació tàctica (*Focus Fire* i *Protect*) per ser realment eficaç.

L’agent heurístic final (**v5 Apex** a Dobles i **v6** a Singles) s’ha consolidat com un baseline de referència molt fort, superant tots els baselines nadius de la llibreria *poke-env* i competint fre a fre amb agents d’estat de l’art com els de *Pokechamp*.

En conclusió, aquest treball ha establert les bases per a la investigació avançada en VGC, proporcionant tant els agents de referència com el pipeline d’entrenament necessari per a futurs desenvolupaments en l’àmbit de la intel·ligència artificial aplicada a jocs d’estratègia complexos.

Bibliografia

- [1] Christopher Berner et al. “Dota 2 with Large Scale Deep Reinforcement Learning”. A: *arXiv preprint arXiv:1912.06680* (2019). DOI: <https://doi.org/10.48550/arXiv.1912.06680>.
- [2] Greg Brockman et al. “OpenAI Gym”. A: *arXiv preprint arXiv:1606.01540* (2016).
- [3] Noam Brown i Tuomas Sandholm. “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals”. A: *Science* 359.6374 (2018), pàg. 418-424. DOI: <https://doi.org/10.1126/science.aao1733>.
- [4] Noam Brown i Tuomas Sandholm. “Superhuman AI for multiplayer poker”. A: *Science* 365.6456 (2019), pàg. 885-890. DOI: <https://doi.org/10.1126/science.aay2400>.
- [5] Cameron B. Browne et al. “A Survey of Monte Carlo Tree Search Methods”. A: vol. 4. 1. 2012, pàg. 1-43. DOI: <https://doi.org/10.1109/TCIAIG.2012.2186810>.
- [6] Bulbapedia Contributors. *Pokémon (species) — Bulbapedia*. Enciclopèdia completa de la franquícia Pokémon. 2024. URL: [https://bulbapedia.bulbagarden.net/wiki/Pok%C3%5C%A9mon_\(species\)](https://bulbapedia.bulbagarden.net/wiki/Pok%C3%5C%A9mon_(species)).
- [7] Bulbapedia Contributors. *Statistic — Bulbapedia, The community-driven Pokémon encyclopedia*. [Online; accessed 4-March-2026]. 2024. URL: <https://bulbapedia.bulbagarden.net/wiki/Stat>.
- [8] Murray Campbell, A.Joseph Hoane i Feng-hsiung Hsu. “Deep Blue”. A: *Artificial Intelligence* 134.1 (2002), pàg. 57-83. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1). URL: <https://www.sciencedirect.com/science/article/pii/S0004370201001291>.
- [9] Wolfe Glick. *WolfeyVGC — Pokémon VGC Educational Content*. <https://www.youtube.com/@WolfeyVGC>. Doble campió del món de VGC (2016, 2019). 2024.
- [10] Donald E. Knuth i Ronald W. Moore. “An analysis of alpha-beta pruning”. A: vol. 6. 4. Elsevier, 1975, pàg. 293-326. DOI: [https://doi.org/10.1016/0004-3702\(75\)90019-3](https://doi.org/10.1016/0004-3702(75)90019-3).
- [11] Yann LeCun, Yoshua Bengio i Geoffrey Hinton. “Deep learning”. A: *Nature* 521.7553 (2015), pàg. 436-444. DOI: <https://doi.org/10.1038/nature14539>.
- [12] Champion Lee et al. *Pokechamp: A Collection of Pokémon Battle AI Agents*. <https://github.com/champion-lee/pokechamp>. Inclou agents basats en MCTS, Minimax, Xarxes Neuronals i heurístiques avançades. 2024.
- [13] Guangcong Luo i Smogon Community. *Pokémon Showdown!* <https://pokemonshowdown.com/>. 2011.
- [14] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. A: *Nature* 518.7540 (2015), pàg. 529-533. DOI: <https://doi.org/10.1038/nature14236>.
- [15] Ollama Team. *Ollama: Get up and running with large language models locally*. <https://ollama.com/>. 2023.
- [16] Antonin Raffin et al. *Stable-Baselines3 Contrib: Reliable Reinforcement Learning Implementations*. <https://github.com/Stable-Baselines-Team/stable-baselines3-contrib>. Inclou MaskablePPO per a entorns amb action masking. 2021.

- [17] Haris Sahovic. *poke-env: A Python Interface for Creating Pokémon Showdown Bots*. <https://github.com/hsahovic/poke-env>. 2020. URL: <https://github.com/hsahovic/poke-env>.
- [18] John Schulman et al. “Proximal Policy Optimization Algorithms”. A: *arXiv preprint arXiv:1707.06347* (2017). DOI: <https://doi.org/10.48550/arXiv.1707.06347>.
- [19] David Silver et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. A: *Science* 362.6419 (2018), pàg. 1140-1144. DOI: <https://doi.org/10.1126/science.aar6404>.
- [20] David Silver et al. “Mastering the game of Go with deep neural networks and tree search”. A: *Nature* 529.7587 (2016), pàg. 484-489. DOI: <https://doi.org/10.1038/nature16961>.
- [21] Richard D. Smallwood i Edward J. Sondik. “The optimal control of partially observable Markov processes over a finite horizon”. A: *Operations Research* 21.5 (1973), pàg. 1071-1088. DOI: <https://doi.org/10.1287/opre.21.5.1071>.
- [22] Smogon University. *Damage Formula*. [Online; accessed 4-March-2026]. 2024. URL: https://www.smogon.com/dp/articles/damage_formula.
- [23] Smogon University. *Smogon Strategy Pokédex*. <https://www.smogon.com/>. Comunitat principal d’estratègia Pokémon competitiva. 2024.
- [24] Richard S. Sutton i Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book.html>.
- [25] The pandas development team. *pandas-dev/pandas: Pandas*. Vers. latest. Febr. de 2020. DOI: 10.5281/zenodo.3509134. URL: <https://doi.org/10.5281/zenodo.3509134>.
- [26] The Pokémon Company International. *History of the Video Game Championships*. <https://www.pokemon.com/us/play-pokemon/about/vgc/>. 2024.
- [27] The Pokémon Company International. *Pokémon World Championships Prize Pool and Awards*. <https://www.pokemon.com/us/play-pokemon/worlds/2024/vgc-competitor-info/>. 2024.
- [28] Oriol Vinyals et al. “Grandmaster level in StarCraft II using multi-agent reinforcement learning”. A: *Nature* 575.7782 (2019), pàg. 350-354. DOI: <https://doi.org/10.1038/s41586-019-1724-z>.
- [29] John Von Neumann i Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 2007. URL: <https://press.princeton.edu/books/paperback/9780691130613/theory-of-games-and-economic-behavior>.
- [30] An Yang et al. *Qwen2 Technical Report*. 2024. arXiv: 2407.10671 [cs.CL].